STUDENT GUIDE

FACULTY OF INFORMATION TECHNOLOGY

Programming 742

RICHFIELD

Registered with the Department of Higher Education as a Private Higher Education Institution under the

Higher Education Act, 1997.

Registration Certificate No. 2000/HE07/008

FACULTY OF INFORMATION TECHNOLOGY

LEARNER GUIDE

MODULE: PROGRAMMING 742

(ASP.NET CORE MVC) (2nd SEMESTER)

PREPARED ON BEHALF OF

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (PTY) LTD

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (Pty) Ltd

Registration Number: 2000/000757/07

All rights reserved; no part of this publication may be reproduced in any form or by any means, including photocopying machines, without the written permission of the Institution.

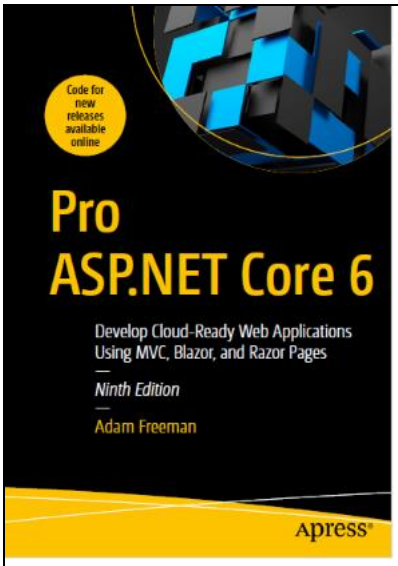
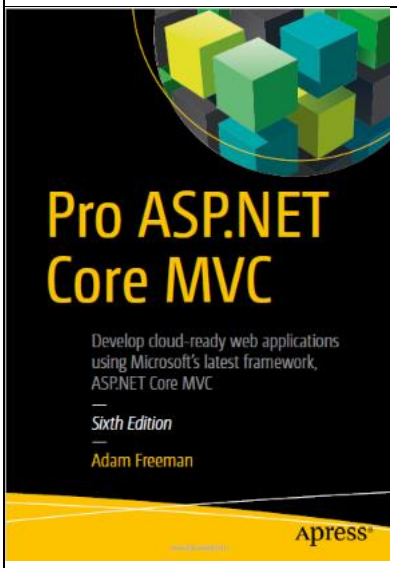
	Freeman, A., 2022. *Pro ASP.NET Core 6: Develop cloud-ready web applications using MVC, Blazor, and Razor Pages*. 9th ed. Berkeley, CA: Apress.
	Freeman, A., 2018. Pro ASP.NET Core MVC: Develop cloud-ready web applications using Microsoft's latest framework, ASP.NET Core MVC. 6th ed. Berkeley, CA: Apress.

Table of Contents

1. Introduction to ASP.NET Core MVC 8
1.1 Introduction to Web Development Paradigms 8
1.1.1 Evolution of Web Development 8
1.1.2 The Need for MVC Architecture 9
1.2 Understanding ASP.NET Core and the .NET Ecosystem 9
1.2.1 The .NET Evolution 9
1.3 Overview of ASP.NET Core Architecture 10
1.4 The Model-View-Controller (MVC) Pattern 11
1.4.1 Model 12
1.4.2 View 12
1.4.3 Controller 13
1.4.4 Request Processing Flow 14
1.4.5 Built-in Features and Capabilities 14
1.5 Setting Up Your Development Environment 15
1.5.1 Installing Visual Studio Community 2022 15
1.6 Creating Your First ASP.NET Core MVC Project 17
1.7 Understanding the Project Structure 19
1.8 Creating an ASP.NET Core MVC Project 22
1.8.1 Anatomy of a Default ASP.NET Core MVC Project 22
1.8.2 Request Lifecycle in ASP.NET Core MVC 22
1.8.3 Scaffolding 23
1.9 Chapter Summary 23
1.10 Review Questions 24
2. Working with Models and Entity Framework Core in ASP.NET Core MVC 25
2.1 Understanding the Role of Models in ASP.NET Core MVC 25
2.2 Defining C# Classes and Modeling Relationships with Entity Framework Core 26
2.2.1 Understanding Entity Framework Core (EF Core) 26
2.2.2 Defining C# Classes and Modeling Relationships with Entity Framework Core 27
2.3 Utilizing Data Annotations and Custom Validation 30
2.4 Understanding the role of the ApplicationDbContext class and Migrations in an ASP.NET Core MVC Application 32

2.4.1 Anatomy of a DbContext Class 32
2.4.2 Configuring the Database Connection 34
2.4.3 Executing Database Migrations using Package Manager Console 36
2.5 Chapter Summary 37
2.6 Review Questions 39
3. Working with Controllers in ASP.NET Core MVC 40
3.1 Understanding the Controller 40
3.1.1 Understanding Dependency Injection 41
3.1.2 Action Methods 42
3.2 EF Core CRUD Operations 45
3.3 Chapter Summary 51
3.4 Review Questions 52
4. Building the UI with Razor Views 53
4.1 What is Razor? 53
4.2 Project Organization & Naming Conventions 56
4.2.1 The Default View Discovery Path 56
4.3 Scaffolding Controllers with Views using EF Core 58
4.4 Strongly-Typed Views – How the View "Receives" the Model 62
4.5 ViewData, ViewBag & TempData – Passing Auxiliary Data 65
4.5.1 Creating a SelectList 66
4.5.2 TempData for User Feedback 68
4.6 Razor Views with JS and CSS 70
4.7 HTML Helpers & Tag Helpers 80
4.8 Tailoring Data for the UI using ViewModels 82
4.9 Chapter Summary 90
4.10 Review Questions 91
5. User Authentication and Role-Based Access Control (RBAC) 93
5.1 Authentication Types 93
5.1.1. Individual Accounts (ASP.NET Core Identity) 93
5.1.2. Microsoft Identity Platform (Microsoft Entra ID / Azure AD) 94
5.1.3. Windows Authentication 94
5.2 Role-Based Access Control (RBAC) 94
5.2.1 ASP.NET Core MVC Identity and RBAC Setup: 95

5.3 Chapter Summary 111
5.4 Review Questions 114
Chapter 6 Extending ASP.NET Core MVC with APIs 115
6.1 What is an API? 115
6.2 RESTful Principles 116
6.3 Creating an API Controller 116
6.3.1 Testing Your API 118
6.3.2 Consuming the API from Client-Side JavaScript 120
6.3.3 Sharing Models and Logic 121
6.3.4 API Versioning 123
6.4 Chapter Summary 123
6.5 Review Questions 124

The Information Technology (IT) qualification at Richfield College is a dynamic and future-focused program designed to equip students with advanced technical, analytical, and problem-solving skills. At the core of the qualification is a commitment to academic excellence, industry alignment, and innovation, fostering graduates who are proficient in addressing modern technological challenges. This qualification strategically integrates theoretical knowledge with practical applications, preparing students for various roles in the IT sector. The IT program is structured to address the growing complexity of the evolving technological landscape.

The Higher certificate in Information Technology (HCIT) program is a foundational stepping-stone for students who wish to pursue further studies or enter the workforce. Graduates of this program are well-prepared to articulate to the Diploma in IT (DIT) or the Bachelor of Science in IT (BSc IT) qualifications, providing a seamless transition for those seeking to deepen their knowledge and skills in specialized IT areas. Additionally, the program equips students with the essential competencies for entry-level IT roles such as IT Support Technicians, Junior Web/ System Developers, IT Administrators, etc.

The Diploma in Information Technology (DIT) is a comprehensive and practical program designed to build a strong foundation in IT principles while equipping students with the hands-on skills required to meet industry demands. Focused on both theoretical knowledge and applied learning, this qualification prepares students for intermediate-level roles in IT and serves as a stepping-stone for further academic progression or specialization. Graduates of this program are well-prepared to articulate to the Bachelor of Science in IT (BSc IT) qualification. The curriculum covers programming, networking, database management, system analysis etc., ensuring graduates possess the competencies to solve real-world IT challenges effectively.

The Bachelor of Science in IT (BSc IT) program is structured to address the growing complexity of the evolving technological landscape. Through carefully curated modules, students gain a deep understanding of software development, database management, cloud computing, cybersecurity, IT management, artificial intelligence, machine learning, networking etc. Graduates of this program are well-prepared to articulate to the Bachelor of Science Honours in IT qualification. The curriculum is designed to bridge the gap between academic learning and real-world applications, thus fostering innovation and an entrepreneurial mindset. Students are encouraged to participate in research and practical learning.

The programming focus within the IT qualification exemplifies academic innovation and professional alignment. By integrating a diverse range of programming languages with practical application, the curriculum prepares students to excel in the rapidly evolving tech industry. The program aligns with industry courses from globally recognized leading tech giants, such as Oracle, AWS, IBM, etc ensures that graduates possess the credentials to validate their expertise in software development and cloud-based technologies. This blend of foundational knowledge, practical experience, and industry-standard courses prepares

students for immediate employment and establishes a strong basis for long-term career advancement in software development.

Programming 742 builds upon the foundational programming skills acquired in Programming 741, transitioning students from C# fundamentals to advanced web development using ASP.NET Core and the Model-View-Controller (MVC) architecture. The module introduces learners to the core principles of building cloud-ready web applications, including routing, middleware, dependency injection, and Razor Pages. Students gain hands-on experience in structuring modern web applications, implementing user interfaces, managing backend logic, and integrating databases. Through practical projects and real-world scenarios, learners develop the ability to design, develop, and deploy scalable, secure, and maintainable ASP.NET applications. This module equips students with the technical expertise needed to advance in full-stack development and enterprise-level software engineering.

1. Introduction to ASP.NET Core MVC



Learning Outcomes:

By the end of this chapter, students should be able to:

- Understand the core concepts of ASP.NET Core MVC
- Identify the key differences between ASP.NET Core and previous versions of ASP.NET
- Describe the Model-View-Controller (MVC) design pattern
- Understand Razor syntax and views
- Understand the routing system in ASP.NET Core MVC
- Understand Dependency Injection in ASP.NET Core MVC
- Recognize the importance of security features like XSS prevention in ASP.NET Core MVC
- Understanding of form validation and error handling
- Understand scaffolding in ASP.NET Core MVC

1.1 Introduction to Web Development Paradigms

1.1.1 Evolution of Web Development

Web development has undergone significant transformations since the early days of static HTML pages. The progression from server-side includes, through Web Forms, to modern MVC architectures represents a continuous effort to address scalability, maintainability, and testability challenges in web application development.

The traditional Web Forms model, while revolutionary for its time, attempted to abstract the stateless nature of HTTP by providing a desktop application-like programming model. However, this abstraction came with significant limitations:

- **Limited control over HTML output**: Web Forms generated HTML automatically, making it difficult to produce semantic, standards-compliant markup
- **Page lifecycle complexity**: The intricate page lifecycle and ViewState management created performance bottlenecks

- **Testing challenges**: The tight coupling between UI and business logic made unit testing extremely difficult
- **Scalability concerns**: ViewState and postback models hindered scalability in high-traffic scenarios

1.1.2 The Need for MVC Architecture

The Model-View-Controller (MVC) pattern emerged as a solution to these challenges, providing:

- Separation of concerns: Clear distinction between data (Model), presentation (View), and application logic (Controller)
- Testability: Loose coupling enables comprehensive unit testing strategies
- Flexibility: Direct control over HTML output and HTTP behaviour
- Scalability: Stateless architecture aligns with HTTP's inherent stateless nature

1.2 Understanding ASP.NET Core and the .NET Ecosystem

1.2.1 The .NET Evolution

Microsoft's .NET ecosystem has undergone substantial restructuring to address modern development requirements. The evolution from .NET Framework through .NET Core to the unified .NET platform reflects Microsoft's commitment to cross-platform development and open-source principles.

Historical Context

The original .NET Framework, introduced in 2002, was designed primarily for Windows-based applications. ASP.NET Web Forms, as part of this framework, provided a familiar event-driven programming model for developers transitioning from desktop application development.

However, the technology landscape evolved rapidly:

- **Cross-platform requirements** became essential
- **Cloud-native architectures** demanded lightweight, modular frameworks
- **Open-source development** became the industry standard
- **Performance optimization** became critical for modern web applications

The .NET Core Revolution

.NET Core represented a fundamental reimagining of the .NET platform:

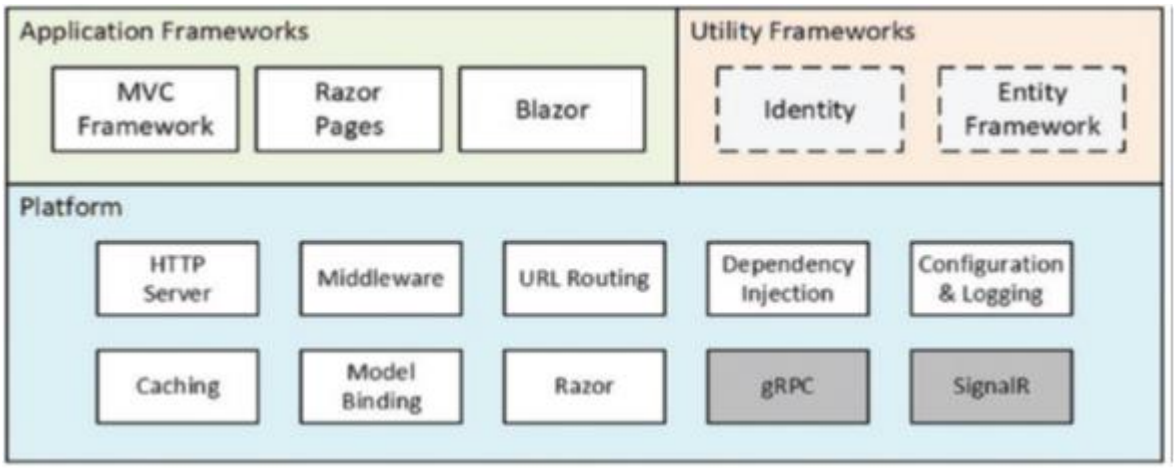


Fig 1.1 *The structure of ASP.NET Core (reference123)*

1.3 Overview of ASP.NET Core Architecture

ASP.NET Core is a powerful, high-performance, cross-platform framework for building modern web applications. It is open-source and designed to provide developers with a lightweight and flexible environment for building dynamic and scalable applications. ASP.NET Core is the successor of the original ASP.NET framework, offering major improvements such as faster performance, reduced overhead, and enhanced security features.

Key features of ASP.NET Core include:

- **Cross-Platform**: ASP.NET Core applications can run on Windows, macOS, and Linux, making it easier to develop and deploy on various operating systems.
- **Modular and Lightweight**: The framework is designed with a modular architecture that allows you to use only the components you need, reducing the footprint and improving performance.
- **Open-Source**: ASP.NET Core is fully open-source, meaning developers can contribute to the project, and it is freely available for anyone to use.
- **Unified Framework**: ASP.NET Core provides a unified approach to building both web APIs and web applications, using the same programming model.

ASP.NET Core provides a modular, high-performance framework for building modern web applications. The architecture consists of several key components:

Core Platform Features

- **HTTP Request Pipeline**: Middleware-based request processing
- **Dependency Injection**: Built-in IoC container for managing dependencies
- **Configuration System**: Unified configuration from multiple sources

- **Logging Framework**: Structured logging with multiple providers
- **Hosting Model**: Flexible hosting options including Kestrel, IIS, and cloud platforms

Application Frameworks

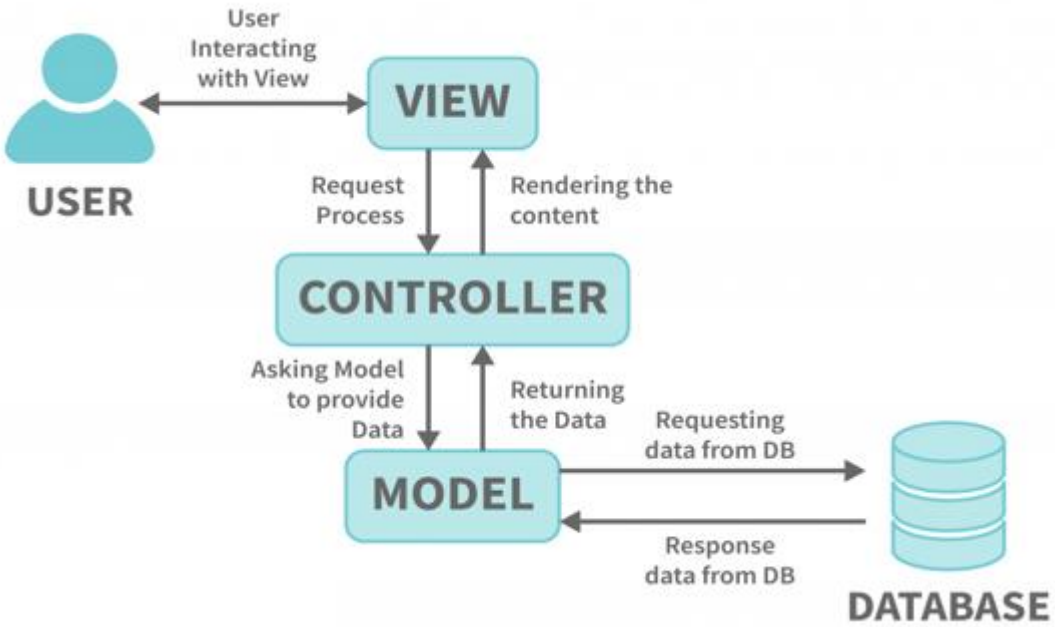
ASP.NET Core supports multiple application development models:

1. **MVC Framework**: Traditional Model-View-Controller pattern
2. **Razor Pages**: Page-based development model
3. **Blazor**: Component-based UI development with C#
4. **Web API**: RESTful service development

1.4 The Model-View-Controller (MVC) Pattern

The Model-View-Controller (MVC) design pattern is a core architectural concept in ASP.NET Core MVC. It separates application logic into three interconnected components: Model, View, and Controller, fostering a clean separation of concerns. This organization simplifies development, testing, and maintenance. The separation of concerns enables:

- **Maintainability**: Changes to one component minimally impact others
- **Testability**: Individual components can be tested in isolation
- **Reusability**: Components can be reused across different contexts
- **Scalability**: Development teams can work on different components simultaneously



1.4.1 Model

Model

The Model represents the data, business logic, and rules of the application. It interacts with the database and performs core operations.

RESPONSIBILITIES

- Accesses and modifies the database.
- Implements business rules and logic.
- Returns data to the Controller.

DOMAIN MODEL

Product.cs

This class represents the core data and includes a business logic method `IsExpensive()`.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string Description { get; set; }
    public DateTime CreatedDate { get; set; }

    // Business logic method
    public bool IsExpensive()
    {
        return Price > 1000;
    }
}
```

VIEW MODEL

ProductListViewModel.cs

Tailored specifically for the View, containing metadata like pagination and search terms.

```
public class ProductListViewModel
{
    public IEnumerable<Product> Products { get; set; }
    public int TotalCount { get; set; }
    public int CurrentPage { get; set; }
    public string SearchTerm { get; set; }
}
```

1.4.2 View

View

The View handles the presentation layer, generating the user interface (UI) displayed to the client.

RESPONSIBILITIES

- Displays data provided by the Model.
- Contains logic for rendering dynamic content.

VIEW LAYER SPECIFICATION

Index.cshtml

Dynamic Rendering

The View receives the **ViewModel** and iterates through the product collection.

It uses conditional logic to highlight premium items, keeping the UI reactive to data changes.

```
@model ProductListViewModel

<h2>Product Catalog</h2>

<div class="search-section">
    <form method="get">
        <input type="text" name="searchTerm" value="@Model.SearchTerm" />
        <button type="submit">Search</button>
    </form>
</div>
```

```razor
<div class="product-grid">
    @foreach (var product in Model.Products)
    {
        <div class="product-card">
            <h3>@product.Name</h3>
            <p class="price">$@product.Price</p>
            <p>@product.Description</p>

            @if (product.IsExpensive())
            {
                <span class="premium-badge">Premium</span>
            }
        </div>
    }
</div>

<div class="pagination">
    <p>Showing @Model.Products.Count() of @Model.TotalCount products</p>
</div>
```

1.4.3 Controller

Controller

Acts as the intermediary. It handles user input, processes it, and determines the appropriate response.

RESPONSIBILITIES

- Receives HTTP requests from the client.
- Interacts with the Model to manipulate data.
- Chooses the appropriate View to render.

CONTROLLER ORCHESTRATION

ProductController.cs

Traffic Control

The Controller handles incoming HTTP requests, interacts with the Service layer, and maps data to the **ViewModel**.

It is responsible for deciding which **View** to return or whether to trigger a **404 Not Found**.

```razor
[Route("products")]
public class ProductController : Controller
{
    private readonly IProductService _productService;

    public ProductController(IProductService productService)
    {
        _productService = productService;
    }

    [HttpGet]
    public async Task<IActionResult> Index(string searchTerm, int page = 1)
    {
        var products = await _productService.GetProductsAsync(searchTerm, page);
        var totalCount = await _productService.GetProductCountAsync(searchTerm);

        var viewModel = new ProductListViewModel
        {
            Products = products,
            TotalCount = totalCount,
            CurrentPage = page,
            SearchTerm = searchTerm
        };

        return View(viewModel);
    }
```

```
    [HttpGet("{id}")]
    public async Task<IActionResult> Details(int id)
    {
        var product = await _productService.GetProductByIdAsync(id);

        if (product == null)
        {
            return NotFound();
        }

        return View(product);
    }
}
```

1.4.4 Request Processing Flow

The MVC request processing follows a well-defined sequence from the moment a user clicks a link to the final HTML render:

1. HTTP Request → Routing Engine
2. Routing Engine → Controller Resolution
3. Controller → Action Method Execution
4. Action Method → Model Interaction
5. Model → Data Processing/Retrieval
6. Controller → View Selection
7. View → Razor Engine Processing
8. Razor Engine → HTML Generation
9. HTML Response → Client Browser

This flow ensures clear separation of concerns while maintaining efficient request processing

1.4.5 Built-in Features and Capabilities

Routing and URL Mapping

Routing is the process of mapping incoming requests to controller actions. ASP.NET Core MVC has a flexible routing system that allows developers to define custom routes using attributes or convention-based patterns. This enables clean and readable URLs for your application.

Razor Views and Syntax

Razor is a view engine that enables developers to write dynamic HTML using C# code. Razor syntax allows you to mix HTML markup with C# code seamlessly, making it easier to create dynamic pages. Razor views are the foundation of how HTML pages are generated in ASP.NET Core MVC applications.

Dependency Injection

ASP.NET Core MVC includes built-in support for Dependency Injection (DI), a technique that helps to manage dependencies between objects. With DI, components like services,

repositories, and controllers are automatically injected into classes, making them more testable and loosely coupled.

Cross-Site Scripting (XSS) Prevention

ASP.NET Core MVC has built-in protection against XSS attacks by automatically encoding user-generated content when it is rendered in views. This ensures that user input is sanitized and prevents malicious content from being executed in the browser.

Built-in Validation and Error Handling

ASP.NET Core MVC includes comprehensive support for form validation using data annotations and model binding. It also offers built-in features for handling and displaying error messages, making it easier to build robust web applications.

1.5 Setting Up Your Development Environment

Successful ASP.NET MVC development requires a properly configured development environment. The following components are essential:

Core Requirements

- **Operating System**: Windows 10/11, macOS, or Linux distribution
- **.NET 8.0 SDK**: Long-term support version for stability
- **Code Editor**: Visual Studio 2022 or Visual Studio Code
- **Database System**: SQL Server LocalDB or SQL Server Express

Recommended Tools

- **Git**: Version control system
- **Postman**: API testing tool
- **Browser Developer Tools**: For debugging client-side code
- **Package Manager**: NuGet for .NET dependencies

1.5.1 Installing Visual Studio Community 2022

Step 1: Download Visual Studio Community 2022

1. Open your browser and go to the Visual Studio Downloads page.
2. Click the Free Download button under Visual Studio Community 2022. (This version is free for individual developers, students, and small teams.)

Step 2: Install the Visual Studio Installer

1. Once the setup file is downloaded, run the installer.
2. The Visual Studio Installer will launch, guiding you through the installation process.

Step 3: Select the Required Workloads

Visual Studio uses workloads to include specific tools and libraries for your development needs. To set up for ASP.NET Core MVC development:

1. In the Workloads tab of the installer, check the following options:
    - ASP.NET and web development: Includes .NET SDK, IIS Express, and tools for web development.
    - Azure Development (Optional): If you plan to deploy your application to Microsoft Azure.
    - Data storage and processing: Database development tools
    - NET desktop development: Additional .NET tools
2. Review the right-hand panel for details about what each workload includes. You can also add:
    - Data Storage and Processing for working with databases.
    - .NET Desktop Development if you plan to build other types of .NET apps.
3. Click Install to begin downloading and installing the selected workloads.

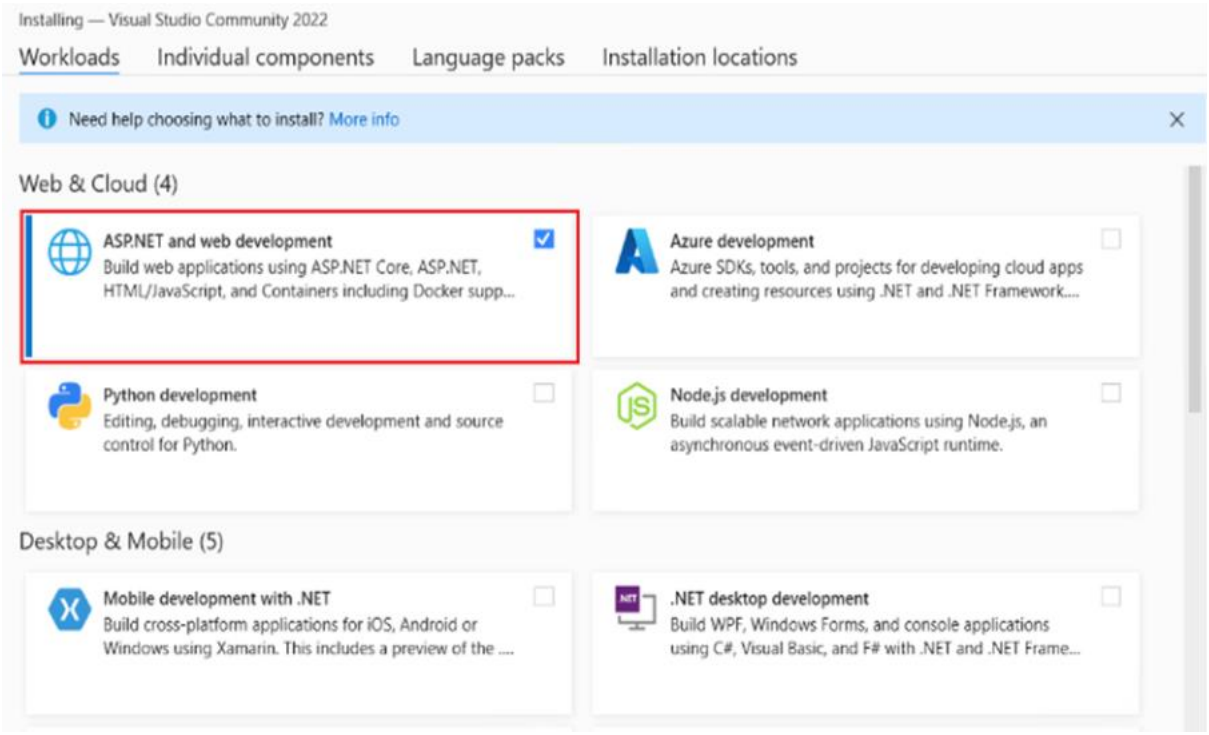
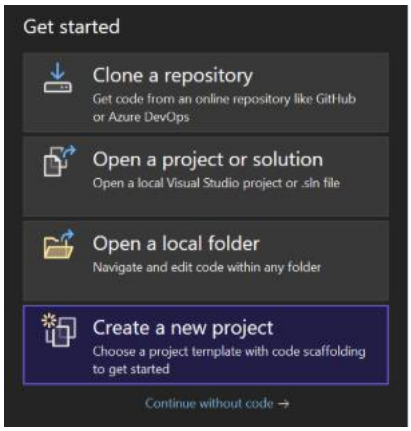
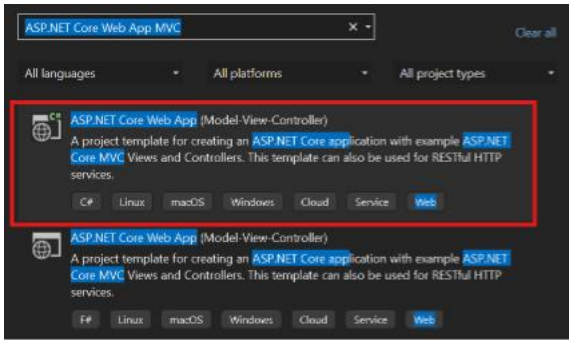


Figure 2A – Workloads Tab

Step 4: Complete the Installation

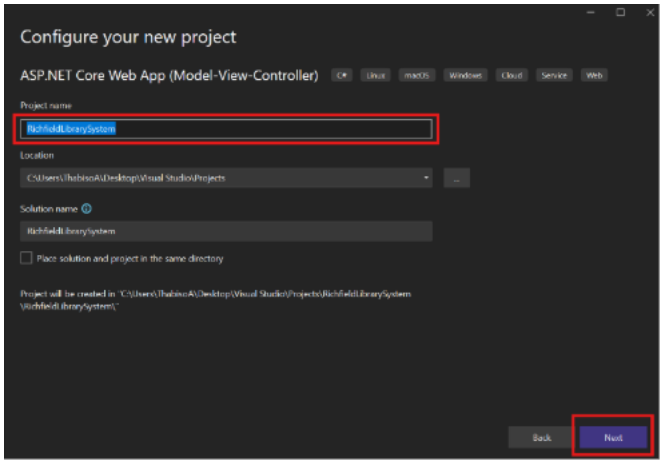
- Once installation is complete, launch Visual Studio by clicking Start Visual Studio.

1.6 Creating Your First ASP.NET Core MVC Project

Step 1: Open Visual Studio • Launch Visual Studio Community 2022. • On the start screen, click Create a new project	
Step 2: Choose a Project Template • In the list of available templates, select ASP.NET Core Web App (Model-View-Controller). • Click Next.	

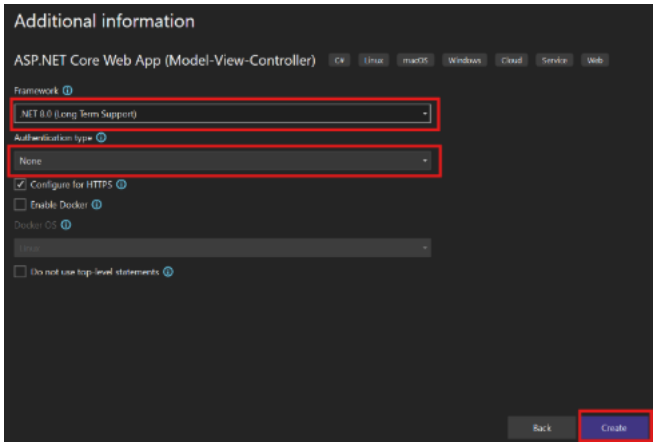
Step 3: Configure Your Project

- Enter a Project Name (e.g., OnlineBookStore).
- Choose a folder in the Location field to save your project files.
- Click Next.



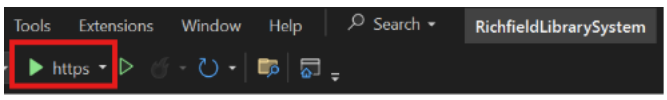
Step 4: Configure Additional Settings

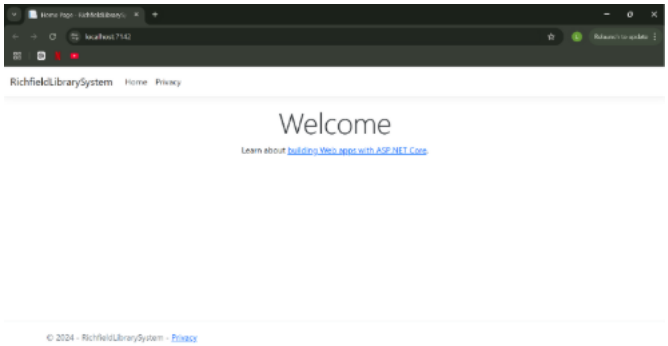
- Select the Target Framework. Choose the latest Long-Term Support (LTS) version of .NET (e.g. .NET 8)
- For Authentication Type, select None for now. This will be covered in a later section.
- Leave other settings as default and click Create.



Step 5: Run Your Application

- After your project is created, click the green Run button (or press Ctrl + F5) to build and run your application.
- Visual Studio will compile the application, start the development server, and open your browser with the

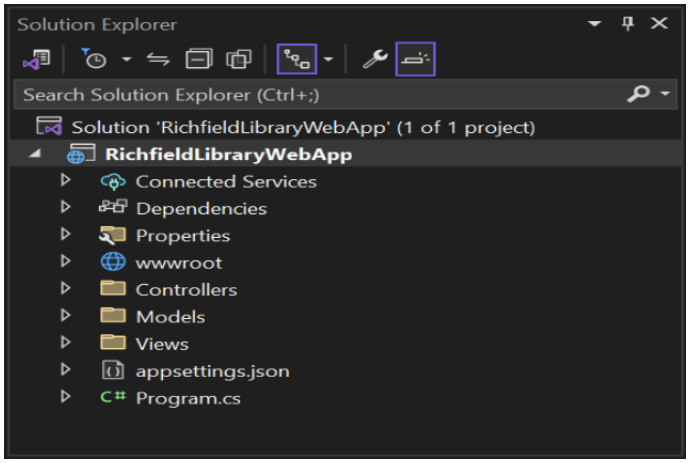




default ASP.NET Core MVC welcome page.	

1.7 Understanding the Project Structure

When you create an ASP.NET Core MVC project, the framework generates a pre-defined structure to organize files and folders. Understanding this structure is crucial to developing robust and maintainable applications.



Connected Services – This folder contains configuration files for integrating external services, such as Azure or other third-party APIs, into your application. It's often used when adding functionality like cloud-based storage, authentication, or analytics.

Dependencies – This node lists all the NuGet packages and project dependencies used in your application. Key packages often include:

- Microsoft.AspNetCore – libraries for MVC framework functionality.
- Microsoft.EntityFrameworkCore – for EF Core integration with your database.

Properties – This folder includes project settings such as launchSettings.json, which defines the environment (e.g., Development, Production) and configurations for running the application locally.

Wwwroot: The wwwroot folder is the designated location for static files, including:

- CSS stylesheets
- JavaScript files
- Images
- Other client-side resources

These files are directly served to the browser.

Controllers

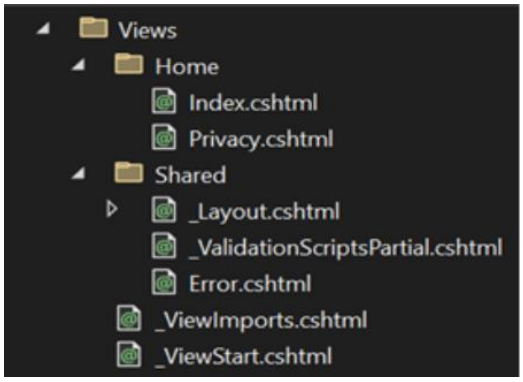
This folder contains the application's **Controllers**, which handle incoming requests, process data, and return responses (e.g., HTML views or JSON data). Controllers are the backbone of the MVC pattern.

Models

The Models folder defines the application's data layer. Here, you can create classes that represent your data (e.g., Book, Member, Category) and use them with Entity Framework Core for database interaction.

Views

The Views folder in an ASP.NET Core MVC project is a critical part of the application. It contains Razor View files (.cshtml) that are responsible for rendering the user interface and presenting data to the user. Views work in tandem with Controllers and Models to implement the MVC pattern.



Key Components of the Views Folder

1. **Home Folder**
    - **Index.cshtml**
      This is typically the default view for the HomeController. It acts as the landing page for your application and is often the first page users see when they visit the root URL of the site.
    - **Privacy.cshtml**
      This is another view associated with the HomeController. It's often used to display information like a privacy policy or other static content.
2. **Shared Folder**: The Shared folder contains views and files that are reusable across multiple pages in the application. These files help maintain consistency and reduce duplication.
    - **_Layout.cshtml:** This file defines the overall structure of your web application (e.g., header, footer, navigation bar). It acts as a template for other views,

ensuring a consistent look and feel across the site. Child views will typically specify this layout using the @layout directive.

- **_ValidationScriptsPartial.cshtml**: This partial view includes scripts for client-side validation, such as jQuery Validation and Unobtrusive Validation. It is often included in forms to handle validation dynamically in the browser.
- **Error.cshtml**: This view is used to display error messages when something goes wrong in the application (e.g., unhandled exceptions). It provides a user-friendly way to inform users about issues.
- **_ViewImports.cshtml**: This file is used to import namespaces, tag helpers, or other configurations that you want available in all views within the folder hierarchy. For example, you might include @addTagHelper, @using directives, or common helper methods.
- **_ViewStart.cshtml**: This file is executed before rendering any Razor View in the folder hierarchy. It allows you to define common settings, such as specifying the layout file (_Layout.cshtml), that apply to all views in the application.

appsettings.json: This is the main configuration file for the application. It stores key application settings and is automatically read by ASP.NET Core during startup. Commonly stored configurations include:

- Database connection strings: Information used to connect to the application's database.
- Logging configurations: Settings for logging levels and outputs, such as console or file logging.
- Application-specific settings: Custom settings needed by your application, like API keys, environment-specific values, or feature flags.

Program.cs: This is the entry point of your application. It sets up the application, including:

- Middleware pipeline
- Dependency injection configuration
- Routing

1.8 Creating an ASP.NET Core MVC Project

To create a new ASP.NET Core MVC project in Visual Studio:

1. Open Visual Studio and select **Create a new project**.
2. Choose **ASP.NET Core Web App (Model-View-Controller)**.
3. Name your project and click **Next**.
4. Select a .NET version (e.g., .NET 8.0 LTS) and click **Create**.

Visual Studio scaffolds the default project structure including:

- **Controllers/**
- **Views/**
- **Models/**
- **wwwroot/**
- **Program.cs** (for configuration)

1.8.1 Anatomy of a Default ASP.NET Core MVC Project

- Controllers/HomeController.cs: Handles HTTP requests and returns views.
- Views/Home/Index.cshtml: Razor view page rendered by the controller.
- Models/: Stores business logic and domain classes.
- wwwroot/: Public assets like CSS, JavaScript, and images.
- Program.cs: Configures services and request pipeline.

1.8.2 Request Lifecycle in ASP.NET Core MVC

1. The user sends an HTTP request.
2. Middleware processes the request.
3. Routing determines which controller and action to invoke.
4. The controller interacts with the model and selects a view.
5. The view renders HTML and returns it to the client.

Example: HomeController



Example: Index.cshtml



1.8.3 Scaffolding

Scaffolding is a code-generation feature that helps developers quickly generate boilerplate code for CRUD operations (Create, Read, Update, Delete) using models. This feature can save time and effort during the development process.

1.9 Chapter Summary

In this opening chapter, we explored the transformation of web development and the architectural shifts that led to the modern .NET ecosystem. We moved from historical development paradigms to the robust, cross-platform reality of **ASP.NET Core**, establishing a firm foundation for building scalable web applications.

We covered the following important concepts**:**

- **The MVC Pattern:** We dissected the Model-View-Controller architecture, identifying the **Model** as the data logic, the **View** as the presentation layer, and the **Controller** as the orchestrator of user requests.

- **The Request Lifecycle:** You learned that a request is not a simple "straight line," but a sophisticated journey—starting at the **Routing Engine**, passing through **Action Methods**, and finally being rendered into HTML via the **Razor Engine**.
- **Ecosystem & Tooling:** We transitioned from theory to practice by setting up **Visual Studio 2022** and exploring the built-in capabilities of the framework, such as **Dependency Injection** and **Validation**, which promote clean, maintainable code.
- **Project Anatomy:** By creating your first project, you gained an understanding of the default file structure, including the role of Program.cs, the wwwroot folder, and how **Scaffolding** can be used to accelerate the development of CRUD operations.

With the environment configured and the core architecture understood, you are now prepared to dive deeper into the specific mechanics of data handling and user interface design.

1.10 Review Questions

1. Define the specific roles of the **Model**, **View**, and **Controller** within the MVC pattern.
2. List four ways the separation of concerns in MVC simplifies the software development lifecycle
3. Outline the nine-step sequence of an MVC request processing flow, starting from the initial HTTP Request to the final HTML Response.
4. Explain how Razor syntax facilitates the creation of dynamic web pages.
5. How does ASP.NET Core MVC provide built-in protection against Cross-Site Scripting (XSS) attacks?
6. Based on the default project structure, what is the role of the wwwroot folder?
7. How does the "Scaffolding" feature assist developers in modern web application projects?
8. In a basic HomeController, which IActionResult method is typically used to render the default landing page of an application?
9. Explain the significance of the appsettings.json file and list two common types of configurations stored within it.
10. What is the role of the Program.cs file, and what key components of the application does it configure?

2. Working with Models and Entity Framework Core in ASP.NET Core MVC



Learning Outcomes:

By the end of this chapter, students should be able to:

- Understand the Role of Models in ASP.NET Core MVC
- Define and create C# classes that represent both application data and relational database schemas.
- Utilize Data Annotations for Validation
- Explain the role of the DbContext and how it serves as the bridge between the MVC application and the database.
- Use EF Core conventions and Data Annotations to map C# objects to database tables, including primary keys and relationships.
- Execute Database Migrations via Package Manager Console to synchronize the database schema with changes made to the Domain Models.

2.1 Understanding the Role of Models in ASP.NET Core MVC

In the **Model-View-Controller (MVC)** design pattern, **models** represent the application's data and business logic. They are responsible for managing data operations, interacting with the database, and enforcing validation rules to ensure data integrity. Models in ASP.NET Core MVC typically fall into two categories:

- **Domain Models:** These are C# classes that represent the data stored in your database. They define the properties (columns) and relationships (tables) of your application's data.
- **ViewModels:** These are specialized classes used specifically to pass data from a Controller to a View. They often combine data from multiple domain models or include extra properties (like dropdown list options) that aren't stored in the database.

The Model's primary roles include:

- **Data Representation:** Defining what a "Product," "User," or "Order" looks like in the system.

- **Data Validation:** Ensuring that data entered by users meets specific business rules before it is saved.
- **Business Logic:** Containing the rules and calculations specific to the application's domain.

2.2 Defining C# Classes and Modeling Relationships with Entity Framework Core

2.2.1 Understanding Entity Framework Core (EF Core)

Entity Framework Core is a modern **Object-Database Mapper (ORM)** for .NET. It serves as an abstraction layer that allows developers to work with a relational database using .NET objects, significantly reducing the amount of manual data-access code (ADO.NET) you need to write.

Core Concepts of EF Core

- **The DbContext (The Bridge):** The DbContext is the most important class in EF Core. It acts as the gateway between your C# application and the database, managing the connection, tracking changes to objects, and translating your requests into SQL.
- **DbSet (The Tables):** Within your DbContext, each DbSet<T> property represents a specific table in the database. For example, DbSet<Student> allows you to query and save instances of the Student class to the "Students" table.
- **POCOs (Plain Old CLR Objects):** EF Core works with standard C# classes (Entities). You don't need to inherit from base classes or implement special interfaces; EF Core uses these classes to understand the "shape" of your data and create the corresponding database schema.

The Workflow: Code-First Migrations

One of the most powerful features of EF Core is the **Code-First** workflow. Instead of designing the database in SQL Server Management Studio, you define your C# classes first.

1. **Define Entities:** Create C# classes with properties and relationships.
2. **Add Migration:** Using the **Package Manager Console**, you run a command (e.g., Add-Migration InitialCreate). EF Core inspects your classes and generates a script that describes the database changes.
3. **Update Database:** You run a second command (Update-Database) which executes that script against your database, ensuring the schema stays in sync with your C# code.

Advantages of EF Core

- Productivity: You can perform complex CRUD (Create, Read, Update, Delete) operations with just a few lines of C# instead of hundreds of lines of SQL and ADO.NET.

- Maintainability: Database changes are tracked through version-controlled migration files.
- Type Safety: Because you are writing C# and LINQ, the compiler catches errors (like misspelling a column name) before the code ever runs.
- Database Provider Support: While often used with SQL Server, EF Core can work with SQLite, PostgreSQL, MySQL, and many others by simply changing the configuration.

2.2.2 Defining C# Classes and Modeling Relationships with Entity Framework Core

To represent application data in a relational database, we use **Entity Framework Core (EF Core)**. This allows us to use a "Code First" approach, where C# classes (Entities) define the database schema. EF Core manages how these classes map to tables and how relationships between those tables are enforced.

Scenario: A University Management System – To understand how EF Core handles data modeling, consider a system managing **Departments**, **Courses**, and **Students**.

1. One-to-Many Relationship (Department to Courses) In EF Core, a one-to-many relationship is defined by a collection in the "one" side and a foreign key property in the "many" side. For example, one department can host many courses, but each course belongs to only one department.

Department.cs

The "One" Side

In this scenario, a **Department** acts as the parent. It can host multiple courses.

Key Features:

- Primary Key identification
- Navigation collection
- Automatic relationship mapping

```
public class Department
{
    // EF Core identifies 'Id' or 'ClassNameId' as PK
    public int DepartmentId { get; set; }
    public string Name { get; set; }

    // Navigation Property: One Department has Many Courses
    public ICollection<Course> Courses { get; set; }
}
```

Course.cs

The "Many" Side

Each **Course** is tied to a specific Department via a Foreign Key.

Requirements:

- Explicit Foreign Key property
- Reference Navigation property

```
public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }

    // Foreign Key property
    public int DepartmentId { get; set; }

    // Navigation Property back to parent
    public Department Department { get; set; }
}
```

2. Many-to-Many Relationship (Students to Courses) A student can enroll in many courses, and a course can have many students. EF Core models this by including a collection property on both entities.

Student.cs

The Student Model

A student can enroll in many courses.

Key Features:

- Shared Navigation Properties
- Implicit Join Table creation
- Collection of related entities

```
public class Student
{
    public int StudentId { get; set; }
    public string FullName { get; set; }

    // EF Core Navigation Property: Many Students in Many Courses
    public ICollection<Course> Courses { get; set; }
}
```

Course.cs

The Course Model

To complete the relationship, the **Course** must also contain a collection of Students. We also include the `DepartmentId` to maintain its link to the organizational hierarchy.

Implementation:

- Foreign Key for Department
- Reciprocal ICollection for Students

```
public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }

    // Foreign Key linked to the Department
    public int DepartmentId { get; set; }

    // The reciprocal collection for the Many-to-Many link
    public ICollection<Student> Students { get; set; }
}
```

Implicit vs. Explicit Join Entities

Sometimes you may need to explicitly create a join entity, however this depends on your data requirements and the version of EF Core being used.

- **Implicit Many-to-Many (EF Core 5.0+):** If you only need to link two entities without storing additional metadata, EF Core handles the relationship automatically. By including collection properties on both sides, a hidden join table is created in the database without requiring a C# class.
- **Explicit Join Entity (Required for "Payload" Data):** You must create a join entity, such as Enrollment, if you need to store additional information about the relationship, such as a Grade, EnrollmentDate, or Status. EF Core cannot automatically assign this "payload" data to a hidden table.
- **Legacy Support:** For projects using EF Core 3.1 or earlier, implicit many-to-many is not supported, and you are required to create a join entity manually to link your models.

Enrollment.cs

The Join Entity

The **Enrollment** class acts as the middle-man. It holds foreign keys to both primary entities and any additional data (payload) specific to that relationship.

```
public class Enrollment
{
    public int StudentId { get; set; }
    public Student Student { get; set; }

    public int CourseId { get; set; }
    public Course Course { get; set; }

    // Extra data: Not about a student OR a course, but an ENROLLMENT
    public DateTime EnrollmentDate { get; set; }
    public string Grade { get; set; }
}
```

Student.cs

Student Model

Instead of a list of Courses, the Student now manages a collection of **Enrollments**.

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }

    public ICollection<Enrollment> Enrollments { get; set; }
}
```

Course.cs

Course Model

Similarly, the Course maps back to the **Enrollment** table to see which students are registered.

```
public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }

    public ICollection<Enrollment> Enrollments { get; set; }
}
```

Explicit Mapping

In modern database design, relationships aren't always straightforward. While a standard naming convention like DepartmentId allows Entity Framework Core to automatically detect associations, real-world legacy databases often use unique naming schemas. The following example demonstrates how to maintain full control over your relational mapping by using the **[Key]** attribute to define non-standard primary keys and the **[ForeignKey]** attribute to explicitly link dependent properties to their parent entities.

Explicit Mapping

The [Key] Attribute

By default, EF Core looks for a property named `Id`. Use `[Key]` when your primary key follows a different naming convention or to be explicit.

```
using System.ComponentModel.DataAnnotations;

public class Passport
{
    [Key]
    public int PassportNumber { get; set; }

    public string IssuingCountry { get; set; }
}
```

Relational Dependency

The [ForeignKey] Attribute

Use `[ForeignKey]` to explicitly link a property to a Navigation Property. This is essential when the property name doesn't match standard conventions.

```
using System.ComponentModel.DataAnnotations.Schema;

public class Course
{
    public int CourseId { get; set; }

    // The actual column in the database
    public int Dept_Ref_Id { get; set; }

    // Explicitly link Dept_Ref_Id to this object
    [ForeignKey("Dept_Ref_Id")]
    public Department Department { get; set; }
}
```

2.3 Utilizing Data Annotations and Custom Validation

Data Annotations are attributes applied to model properties that serve a dual purpose: they define database schema constraints (for EF Core) and provide client-side and server-side validation rules for the MVC framework.

When a user submits a form, the **Model Binder** inspects these attributes. If any rule is violated, ModelState.IsValid will return false in the Controller, allowing you to return the user to the form with specific error messages.

Built-in Validators

Common Attributes

Leveraging out-of-the-box validators for emails, ranges, and patterns.

- **[Required]**: Mandatory field check.
- **[EmailAddress]**: Regex-free email validation.
- **[Phone]**: Telephone format validation.
- **[Compare]**: Password confirmation.

```
public class UserRegistrationViewModel
{
    [Required(ErrorMessage = "Full Name is mandatory")]
    [StringLength(100, MinimumLength = 3)]
    public string FullName { get; set; }

    [Required]
    [EmailAddress(ErrorMessage = "Invalid Email")]
    public string Email { get; set; }

    [Required]
    [Phone(ErrorMessage = "Invalid phone number format")]
    public string PhoneNumber { get; set; }
```

```
    [Range(18, 120)]
    public int Age { get; set; }

    [Required]
    [RegularExpression(@"^(?=.*[A-Z]).{8,}$")]
    public string Password { get; set; }

    [Compare("Password")]
    public string ConfirmPassword { get; set; }
}
```

Custom Validation

Defining the Rule

The `ValidationAttribute` base class allows us to override `IsValid` to create any business rule imaginable.

```
public class NotInFutureAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(object value, ValidationContext ctx)
    {
        if (value is DateTime dateTime && dateTime > DateTime.Now)
        {
            return new ValidationResult("Date cannot be in the future");
        }
        return ValidationResult.Success;
    }
}
```

Model: Employee.cs

Applying the Attribute

Once the attribute is defined, we apply it to our Model properties just like the built-in `[Required]` tag.

```
public class Employee
{
    public int Id { get; set; }

    [Required]
    public string Name { get; set; }

    // Applying the custom validation defined above
    [NotInFuture(ErrorMessage = "Hire date cannot be a future date")]
    public DateTime HireDate { get; set; }
}
```

Implementation & Structure

To maintain a professional and scalable codebase, it is best practice to separate your business logic (validation) from your data structures (models).

Project Organization:

- Models/: Contains your standard Entity Framework classes, such as Employee.cs.
- ValidationAttributes/: A dedicated folder for your custom logic classes, like NotInFutureAttribute.cs.
- Controllers/: Where the ModelState.IsValid check occurs during form submission.

Project Architecture

Directory Layout

By placing custom attributes in their own directory, you keep the **Models** folder clean and focused on data properties.

```
DemoMVCApplication
  Controllers
    EmployeeController.cs
  Models
    Employee.cs
  ValidationAttributes
    NotInFutureAttribute.cs
  Views
    Employee
      Create.cshtml
  appsettings.json
  Program.cs
```

2.4 Understanding the role of the ApplicationDbContext class and Migrations in an ASP.NET Core MVC Application

The DbContext is the most vital component in Entity Framework Core. It acts as the primary **bridge** between your C# code and the database, translating object-oriented operations into SQL commands. Think of it as the "Brain" of your data layer—it manages connections, tracks changes to your objects, and ensures data integrity during every transaction.

2.4.1 Anatomy of a DbContext Class

A typical DbContext class is composed of four specific elements that define how your application interacts with the data.

1. **Inheritance from DbContext:** Every context must inherit from EF Core's DbContext. This inheritance provides your class with the built-in functionality required to query and save data.
2. **The Constructor:** The constructor typically accepts a DbContextOptions<TContext> object. This allows the application to "inject" configurations—such as the database provider (SQL Server, SQLite) and the connection string—from Program.cs.
3. **DbSets (The Tables):** A DbSet<TEntity> represents a collection of a specific entity in the context. In database terms, each DbSet property corresponds directly to a **Table** in your database.
4. **The OnModelCreating() Method:**This method is used to configure the model using the **Fluent API**. It is the place where you define complex relationships (like many-to-many join entities), set constraints, and seed initial data.

Fluent API

What is the Fluent API?

The **Fluent API** is a powerful way to configure your domain classes in EF Core using code-based logic within the `OnModelCreating()` method. While Data Annotations are simple tags added to properties, the Fluent API provides a centralized and more capable approach to mapping.

Key Concept: Method Chaining

The term "Fluent" comes from the way methods are "chained" together to read like a sentence. For example:

*"The **Entity** 'Enrollment' **Has a Key** that is a **New** composite of StudentId and CourseId."*

Fluent API vs. Data Annotations

Feature	Data Annotations	Fluent API
Location	In the Model class	In the DbContext class
Composite Keys	❌ Not Supported	✅ Supported (Required)
Data Seeding	❌ Not Possible	✅ .HasData()
Relationships	Simple (1:1, 1:N)	Complex (Many-to-Many)
Clean Code	Clutters Model classes	Keeps Models "POCO"
Control	Basic Constraints	Full DB Customization

Syntax Example

```
// Configuring a Composite Key using Method Chaining
modelBuilder.Entity<Enrollment>()
    .HasKey(e => new { e.StudentId, e.CourseId });
```

Seeding Initial Data

Data seeding is the process of populating a database with an initial set of data. It is essential for:

- Testing: Providing dummy records for UI development.
- Look-up Tables: Populating static lists like "Countries," "Roles," or "Department names" that the application needs to function immediately after deployment.

Implementation Demonstration

ApplicationDbContext.cs

DbContext Anatomy

The Bridge in Action

This class serves as the single point of contact between your C# entities and the SQL database.

- **Inheritance:** Unlocks EF Core's engine.
- **Constructor:** Receives settings from Program.cs.
- **DbSets:** Becomes your SQL Tables.
- **Fluent API:** Handles logic too complex for attributes.

```
using Microsoft.EntityFrameworkCore;

public class ApplicationDbContext : DbContext
{
    // 1. CONSTRUCTOR
    // Injects provider settings (SQL Server) and connection strings
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    // 2. DBSETS
    // These represent the Tables that will be created in the DB
    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }
    public DbSet<Enrollment> Enrollments { get; set; }
```

```
    // 3. ONMODELCREATING
    // Used for Fluent API configuration and seeding data
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Configure Composite Key for many-to-many join table
        modelBuilder.Entity<Enrollment>()
            .HasKey(e => new { e.StudentId, e.CourseId });

        // Configure One-to-Many Relationship (Student to Enrollments)
        modelBuilder.Entity<Enrollment>()
            .HasOne(e => e.Student)
            .WithMany(s => s.Enrollments)
            .HasForeignKey(e => e.StudentId);

        // Seed initial Data
        modelBuilder.Entity<Course>().HasData(
            new Course { CourseId = 1, Title = "Advanced C#" },
            new Course { CourseId = 2, Title = "EF Core Fundamentals" }
        );
    }
}
```

2.4.2 Configuring the Database Connection

Before the DbContext can communicate with a database, it needs two things: a **Connection String** (the address and credentials) and **Registration** (telling the application at startup that the service exists).

- **The Connection String (appsettings.json)**
  The connection string is a formatted string that tells EF Core exactly where the database is located and how to access it. Storing it in appsettings.json is a standard practice because it allows you to change databases (e.g., from a local test database to a production server) without recompiling your code.

```
// appsettings.json - Connection Strings Section
{
  "ConnectionStrings": {
    // Connection string name (reference this in Program.cs)
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SchoolDB;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"

    // Alternative: SQL Server with username/password authentication
    // "ProductionConnection": "Server=sqlserver01;Database=SchoolDB;User Id=appuser;Password=P@ssw0rd123;Encrypt=True;TrustServerCertificate=True"

    // Alternative: SQL Server with specific port and timeout
    // "CustomConnection": "Server=localhost,1433;Database=SchoolDB;User Id=sa;Password=Password123;Connect Timeout=30;Encrypt=False"
  },

  // Other configuration sections...
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Server
Database server name/address

Database
Specific database name

Trusted_Connection
Windows Authentication

MultipleActiveResultSets
Allow concurrent queries

LocalDB is a lightweight SQL Server Express edition that runs in user mode. Use `(localdb)\\mssqllocaldb` as the server name for local development. This creates a user instance of SQL Server that doesn't require a full SQL Server installation.

TrustServerCertificate
SSL certificate validation

Encrypt
Enable encryption for data

- **Registering the DbContext (Program.cs)**
  In modern ASP.NET Core, we use **Dependency Injection (DI)** to manage services. We must register our ApplicationDbContext in the Program.cs file. This tells the application to create an instance of the context whenever a Controller requests it, using the connection string we just defined.

```
using Microsoft.EntityFrameworkCore;
using DemoMVCApplication.Data;

// Create the application builder
var builder = WebApplication.CreateBuilder(args);

// Add services to the Dependency Injection container
builder.Services.AddControllersWithViews();
```

```
// REGISTER DBCONTEXT WITH SQL SERVER PROVIDER
// ServiceCollection extension method for DbContext
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
    // Configure DbContextOptions for SQL Server
    options.UseSqlServer(
        // Retrieve connection string from configuration
        builder.Configuration.GetConnectionString("DefaultConnection"),

        // (Optional) Additional SQL Server options
        sqlServerOptions =>
        {
            // Command timeout in seconds (default is 30)
            sqlServerOptions.CommandTimeout(30);
```

```
            // Enable retry on failure (useful for transient errors)
            sqlServerOptions.EnableRetryOnFailure(
                maxRetryCount: 3,
                maxRetryDelay: TimeSpan.FromSeconds(5),
                errorNumbersToAdd: null);

            // (Optional) Enable connection resiliency
            sqlServerOptions.EnableRetryOnFailure();
        }
    );
```

```
    // (Optional) Enable detailed errors and sensitive data logging
    // WARNING: Only for development environment!
    if (builder.Environment.IsDevelopment())
    {
        options.EnableDetailedErrors(true);
        options.EnableSensitiveDataLogging(true);
    }
});
```

```
// Alternative: Simplified registration (most common usage)
// builder.Services.AddDbContext<ApplicationDbContext>(options =>
//     options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// ... rest of middleware configuration (app.UseRouting(), app.MapControllerRoute(), etc.)
```

How it works: When a controller requests ApplicationDbContext via constructor injection, the DI container provides a scoped instance configured with your SQL Server connection. This instance is automatically disposed at the end of the HTTP request, ensuring proper resource cleanup.

Lifetime Options

- **Scoped (default for DbContext):** One instance per HTTP request
- **Transient:** New instance every time it's requested
- **Singleton:** One instance for entire application lifetime

2.4.3 Executing Database Migrations using Package Manager Console

Once your **DbContext** is configured and your models are ready, you must synchronize these C# changes with the actual SQL database. **Migrations** are Entity Framework Core's way of version-controlling your database schema. Instead of manually writing SQL scripts to create tables or add columns, you use the **Package Manager Console (PMC)** to generate and execute these changes.

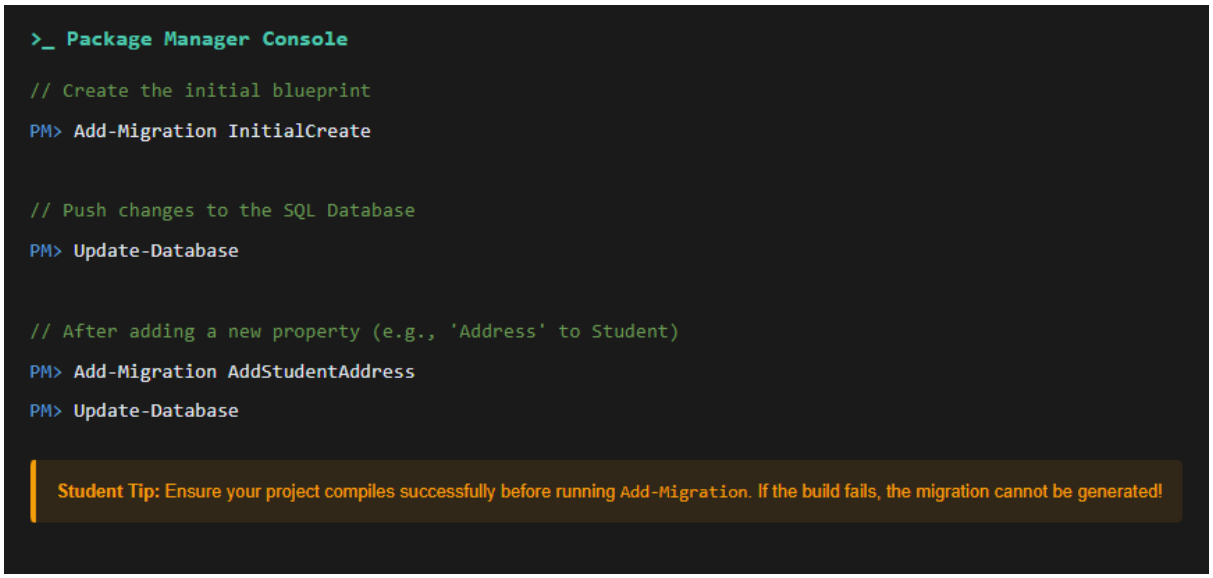
The Migration Workflow

Synchronizing your models with the database follows a strict two-step cycle:

- **Add-Migration:** Scans your DbContext and models, then generates a C# "snapshot" of the changes required.
- **Update-Database:** Translates that C# snapshot into a SQL script and executes it against your database.

- **Remove-Migration**: Deletes the last migration file (use only if not yet updated).
- **Update-Database 0:** Run **Update-Database 0** to revert all applied migrations and return the database to its empty state, then manually delete the files in the Migrations/ folder to start completely fresh. **(NB: Running Update-Database 0 will permanently delete all data stored in your tables. This should only be used in development environments when you are ready to rebuild the database from scratch)**

📍 **WHERE IS THE PMC?**
Go to: Tools → NuGet Package Manager → Package Manager Console



2.5 Chapter Summary

In this chapter, we moved beyond basic C# objects to build a **reliable and scalable** data layer. We explored how the **Model** serves as the single source of truth, dictating both the application's logic and the relational database's structure.

- **The Foundation of Models:** You've learned that Models are the "M" in MVC—the blueprints for your data. By defining domain classes like Student or Employee, you provide EF Core with the template it needs to generate corresponding database tables.
- **Enforcing Data Integrity:** We utilized **Data Annotations** (such as [Required] and [StringLength]) and custom validation logic to ensure data remains consistent. By placing these constraints directly on the Model, we ensure validation happens before data ever touches the database.

- **The DbContext Bridge:** You now understand the DbContext as the central coordinator. It manages database connections, tracks object changes, and acts as the vital link between object-oriented C# code and the relational world of SQL.
- **Configuration and Security:** We mastered the art of connection strings—breaking down components like Server and Trusted_Connection. More importantly, you learned to store these securely in appsettings.json and register the DbContext within the application's service container.
- **Relationship Mapping:** Using both EF Core Conventions and the **Fluent API**, we mapped complex relationships, including primary keys and many-to-many associations via Join Entities like Enrollment.
- **Schema Synchronization:** Finally, you mastered the migration lifecycle. By using Add-Migration and Update-Database, you can now keep your SQL schema in perfect lock-step with your C# code changes.

Now that our data structures are defined and our database is live, we need a way to move that data into the hands of the user. In **Chapter 3**, we shift our focus to the **Controller**—the "brain" of the MVC pattern. You will learn how to use **Dependency Injection** to pull your DbContext into a controller, allowing you to write **Action Methods** that query, create, and update records in real-time. We will explore how the Controller orchestrates the flow of information, taking the models we've built and preparing them for the View.

2.6 Review Questions

1. Describe how a Model class in ASP.NET Core MVC serves a "dual purpose" for both the application's C# logic and the database engine.
2. Explain the difference between using a built-in attribute like [EmailAddress] and creating a custom ValidationAttribute. In what scenario would a custom attribute be mandatory?
3. Define the DbContext. Why is it considered the "gateway" or "bridge" between your code and your persistent storage?
4. Under what specific condition can you let EF Core handle a many-to-many relationship implicitly (without a C# join class), and when must you create an explicit join entity like Enrollment?
5. Look at the following code snippet. Translate this "Method Chain" into a plain English sentence that explains what is happening to the database schema:

```
modelBuilder.Entity<Enrollment>().HasKey(e => new { e.StudentId, e.CourseId });
```

6. Break down each method: Entity<Enrollment>(), HasKey(), and the anonymous type new { e.StudentId, e.CourseId }. What type of key is being created?
7. A developer has just added a new property DateOfBirth to the Employee model. List the exact two commands they must run in the Package Manager Console to ensure the SQL database now contains a DateOfBirth column.
8. Why do we pass DbContextOptions<ApplicationDbContext> options into the ApplicationDbContext constructor? What role does this play in the startup of the application?
9. You need to ensure that every time the database is created, it automatically includes a list of "Departments" (HR, IT, Finance). Which method within the DbContext would you override to implement this, and what is the specific EF Core method used to add the data?
10. Explain the role of the following code in Program.cs and what each part accomplishes:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

11. Why is the DbContext typically registered with a **scoped** lifetime in web applications? What problems could occur if it were registered as a **singleton** instead?

3. Working with Controllers in ASP.NET Core MVC

Learning Outcomes:

By the end of this chapter, students should be able to:

- Explain the structural role of the Controller as the primary intermediary within the MVC pattern.
- Create controller classes that inherit from the Controller base class to access built-in web-handling logic.
- Dependency Injection – Implement the Dependency Inversion Principle by injecting DbContext and ILogger services via the class constructor.
- Configure how URLs map to code using conventional naming and explicit attributes like [HttpPost].
- Distinguish between synchronous and asynchronous (Task<IActionResult>) methods to optimize server performance during I/O operations.
- Understand various return types, including ViewResult, RedirectToActionResult, and status results like NotFound or BadRequest.
- Apply ModelState.IsValid for data validation and utilize the Post-Redirect-Get pattern to ensure a smooth, secure user experience.
- Build complete Create, Read, Update, and Delete logic using methods like ToListAsync(), Add(), and SaveChangesAsync().
- Query Optimization – Enhance performance using FindAsync() for ID lookups and Include() to load related data efficiently.
- Error Handling – Gracefully manage database exceptions and prevent "over-posting" vulnerabilities to maintain application stability and security.

3.1 Understanding the Controller

In ASP.NET Core MVC, a **Controller** is a class that handles incoming HTTP requests (like clicking a link or submitting a form), performs business logic, and returns a response to the user.

The Controller acts as a "Middleman." It:

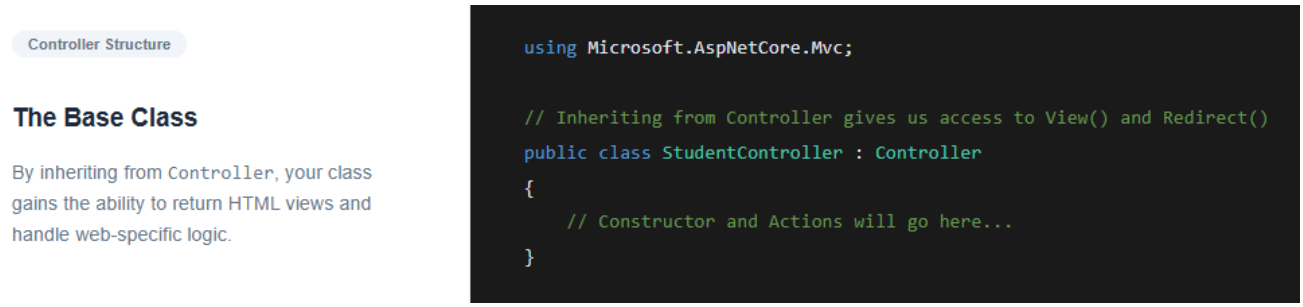
- **Accepts Input:** Reads data from the URL, form fields, or headers.
- **Interacts with the Model:** Uses the DbContext to fetch or save data.
- **Selects a View:** Decides which page the user should see next.

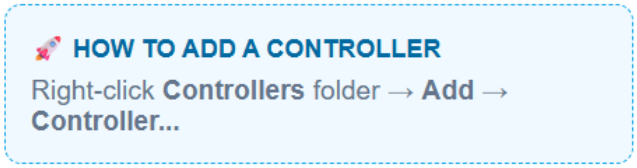
Every controller must inherit from the Microsoft.AspNetCore.Mvc.Controller base class. This inheritance is vital because it provides access to essential MVC features like View(), RedirectToAction(), ModelState, and User session data.

🗀 Controller Location

Controllers are typically placed in the Controllers folder of your project. This follows the convention-over-configuration principle of ASP.NET Core.

- **Controllers/** - Contains all controller classes
- **Views/** - Contains corresponding view files
- **Models/** - Contains model classes





3.1.1 Understanding Dependency Injection

In modern ASP.NET Core development, we rarely create instances of complex objects manually (using the new keyword). Instead, we use **Dependency Injection (DI)**. This is a design pattern where the framework "injects" the tools a controller needs directly into its constructor at the moment the controller is created.

To understand this, let's look at the two primary dependencies used in our StudentController example:

1. The ApplicationDbContext (The Data Dependency)

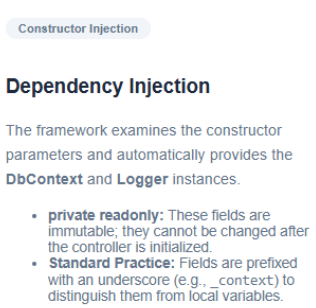
The ApplicationDbContext is your controller's connection to the database.

- **Purpose:** It allows the controller to perform CRUD operations (Create, Read, Update, Delete) on your domain models.
- **Why it's a "Dependency":** The controller depends on the context to fetch data, but it doesn't need to know the database connection string or whether you are using SQL Server or SQLite—the DI container handles those details.

2. The ILogger (The Diagnostic Dependency)

The ILogger<T> is a built-in service used to record events that happen while your application is running.

- **Purpose:** It helps developers track errors, monitor performance, and debug issues in production without stopping the app.
- **Why it's a "Dependency":** Your controller depends on a logger to report its activity, but the controller doesn't need to know *how* the logging works (whether it saves to a text file, a cloud service, or the visual console).




```
public class StudentController : Controller
{
    // Private fields to hold our injected dependencies
    private readonly ApplicationDbContext _context;
    private readonly ILogger<StudentController> _logger;

    // The constructor: The "Injection Site"
    public StudentController(ApplicationDbContext context, ILogger<StudentController> logger)
    {
        _context = context;
        _logger = logger;
    }
```

```
    public async Task<IActionResult> Index()
    {
        // Using the Logger dependency
        _logger.LogInformation("Fetching student list from the database...");

        // Using the DbContext dependency
        var students = await _context.Students.ToListAsync();

        return View(students);
    }
}
```

The Benefits of DI

1. **Maintainability:** If you decide to change your logging system (e.g., from Console to a cloud-based log), you don't have to change a single line of code in your Controllers.
2. **Testability:** You can easily "mock" (fake) the database or the logger when writing unit tests to ensure your logic works correctly.
3. **Clean Code:** The controller remains focused on handling web requests rather than setting up infrastructure (and sometimes even querying the Db).

3.1.2 Action Methods

An **Action Method** is a public method inside a controller that responds to a URL. For example, if a user visits /Student/Index, the Index() action method inside the StudentController is executed.

ActionResult vs. Async Task

Most modern applications use **Asynchronous** actions. This prevents the server from "freezing" while waiting for a database to respond, allowing it to handle more users simultaneously.

- **Synchronous:** public IActionResult Index()
- **Asynchronous:** public async Task<IActionResult> Index() (The industry standard for DB operations).

Common Action Results

Action methods return an IActionResult, which is a "contract" saying: "I will return some kind of web result."

Method	Result Type	Use Case & Description
`return View();`	ViewResult	**Renders the .cshtml page** corresponding to the action name. Looks in `Views/ControllerName/` directory. Can pass a model object as parameter: `return View(model);`
`return View("CustomView");`	ViewResult	**Renders a specific view file** by name. Useful when you want to return a view with a different name than the action method. Example: `return View("Error");`
`return RedirectToAction("Index");`	RedirectToActionResult	**Sends the user to a different action method** (HTTP 302 redirect). Common after form submissions (POST-REDIRECT-GET pattern). Can pass route values: `return RedirectToAction("Details", new { id = 5 });`
`return Redirect("/Home/Index");`	RedirectResult	**Redirects to a specific URL** (absolute or relative). Use when redirecting to external URLs or non-action endpoints. Example: `return Redirect("https://example.com");`
`return NotFound();`	NotFoundResult	**Returns a 404 Not Found error** when a requested resource doesn't exist. Can include an optional message: `return NotFound("User not found");`
`return BadRequest();`	BadRequestResult	**Returns a 400 Bad Request error** when client sends invalid data. Often used with ModelState validation: `return BadRequest(ModelState);`
`return Json(data);`	JsonResult	**Returns JSON-formatted data** for API endpoints. Sets Content-Type to application/json. Example: `return Json(new { id = 1, name = "Test" });`

Action Results

Common Responses

These built-in methods are inherited from the `Controller` base class. They simplify the process of sending the correct HTTP response codes.

💡 **Pro Tip:** Use `RedirectToAction` instead of `Redirect` whenever possible to avoid "hardcoding" URLs.

```
// 1. Return a View (HTML Page)
public IActionResult Details() {
    return View();
}

// 2. Redirect to another Action (Internal)
public IActionResult SaveData() {
    // Logic here...
    return RedirectToAction("Index");
}

// 3. Hard Redirect to a URL
public IActionResult GoHome() {
    return Redirect("/Home/Index");
}
```

```
// 4. Return a 404 Error
public IActionResult GetStudent(int id) {
    if (id == 0) return NotFound();
    return View();
}

// 5. Return a 400 Error (Bad Request)
public IActionResult Update() {
    if (!ModelState.IsValid) return BadRequest();
    return Ok();
}

// 6. Return raw Data (AJAX/API)
public IActionResult GetData() {
    var data = new { Name = "John", Age = 25 };
    return Json(data);
}
```

The Power of async and await

By using **Asynchronous** methods (async Task<IActionResult>), we tell the server: *"Go fetch this data from the database. While you wait for the database to respond, you are free to handle other users' requests."* Once the data arrives, the server resumes where it left off. This makes your application significantly more scalable.

In some action methods you will notice the [Bind] attribute being used. It is used in controller actions to explicitly specify which properties of a model class should be included in model binding. For example:

```
public async Task<IActionResult> Create([Bind("Id","FullName","Email")] Student student)
```

- **Purpose**: It tells the model binder to only bind the properties Id, FullName, and Email from the HTTP request (form data, query string, etc.) to the Student object. Any other properties (e.g., IsAdmin, PasswordHash) are ignored and will retain their default values.

This is also a security measure to prevent **over-posting** (also known as mass assignment) attacks. Without [Bind], an attacker could send extra fields in the request (e.g., IsAdmin=true) and potentially elevate privileges or modify data they shouldn't. By whitelisting only the properties you expect, you protect your model from unintended updates.

3.2 EF Core CRUD Operations

CRUD stands for **C**reate, **R**ead, **U**pdate, and **D**elete. These are the four basic functions of persistent storage. Below is a comprehensive example of a StudentController performing these operations using the _context we injected earlier.

Async CRUD Operations

Complete CRUD Pattern

This controller demonstrates the complete CRUD lifecycle with Entity Framework Core, incorporating best practices for production-ready applications.

💡 **EF Core Magic:** DbContext tracks changes automatically. `Add()`, `Update()`, and `Remove()` only mark entities; `SaveChangesAsync()` executes the actual SQL transaction.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SchoolApp.Data;
using SchoolApp.Models;

namespace SchoolApp.Controllers
{
    [Route("[controller]")]
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly ILogger<StudentController> _logger;

        // Constructor Injection
        public StudentController(
            ApplicationDbContext context,
            ILogger<StudentController> logger)
        {
            _context = context;
            _logger = logger;
        }
```

```
        // GET: Student/Index
        // Displays all students with their enrollments
        [HttpGet]
        public async Task<IActionResult> Index()
        {
            // Eager load related data to avoid N+1 queries
            var students = await _context.Students
                .Include(s => s.Enrollments)
                    .ThenInclude(e => e.Course)
                .AsNoTracking()  // Read-only, better performance
                .ToListAsync();

            return View(students);
        }
```

```
// GET: Student/Details/5
[HttpGet("Details/{id}")]
public async Task<IActionResult> Details(int? id)
{
    if (id == null)
        return BadRequest("Student ID is required");

    var student = await _context.Students
        .Include(s => s.Enrollments)
            .ThenInclude(e => e.Course)
        .FirstOrDefaultAsync(s => s.Id == id);

    if (student == null)
    {
        _logger.LogWarning("Student {Id} not found", id);
        return NotFound();
    }

    return View(student);
}
```

```
// GET: Student/Create
[HttpGet("Create")]
public IActionResult Create()
{
    // Populate ViewBag with select lists if needed
    return View();
}
```

```
[HttpPost("Create")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Name,Email,EnrollmentDate")] Student student)
{
    if (ModelState.IsValid)
    {
        try
        {
            // Add to change tracker
            _context.Add(student);
            await _context.SaveChangesAsync();

            // Success feedback
            TempData["SuccessMessage"] = $"Student {student.Name} created successfully!";

            // POST-REDIRECT-GET pattern
            return RedirectToAction(nameof(Index));
        }
        catch (DbUpdateException ex)
        {
            // Log the full error for debugging
            _logger.LogError(ex, "Error creating student");

            // Add user-friendly error
            ModelState.AddModelError("",
                "Unable to save student. Please try again.");
        }
    }

    // If we got this far, something failed; redisplay form
    return View(student);
}
```

```
// GET: Student/Edit/5
[HttpGet("Edit/{id}")]
public async Task<IActionResult> Edit(int? id)
{
    if (id == null)
        return BadRequest();

    var student = await _context.Students
        .FindAsync(id);  // FindAsync is optimized for PK lookups

    if (student == null)
        return NotFound();

    return View(student);
}
```

```
// POST: Student/Edit/5
[HttpPost("Edit/{id}")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
    [Bind("Id,Name,Email,EnrollmentDate")] Student student)
{
    if (id != student.Id)
        return NotFound();

    if (ModelState.IsValid)
    {
        try
        {
            // Mark entity as modified
            _context.Update(student);
            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Student updated successfully!";

            return RedirectToAction(nameof(Index));
        }
```

```
        catch (DbUpdateConcurrencyException)
        {
            // Handle concurrent updates
            if (!StudentExists(student.Id))
            {
                return NotFound();
            }
            else
            {
                ModelState.AddModelError("",
                    "This student was modified by another user. Please refresh and try again.");
            }
        }
        catch (DbUpdateException ex)
        {
            _logger.LogError(ex, "Error updating student {Id}", id);
            ModelState.AddModelError("",
                "Unable to save changes. Please try again.");
        }
    }

    return View(student);
}
```

```
// GET: Student/Delete/5 (confirmation page)
[HttpGet("Delete/{id}")]
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
        return BadRequest();

    var student = await _context.Students
        .Include(s => s.Enrollments)  // Check for dependent data
        .FirstOrDefaultAsync(s => s.Id == id);

    if (student == null)
        return NotFound();

    return View(student);
}
```

```
// POST: Student/Delete/5 (actual deletion)
[HttpPost("Delete/{id}")]
[ActionName("Delete")]  // Match the GET action name
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    try
    {
        // Find the student - but we don't need full entity for deletion
        var student = await _context.Students.FindAsync(id);

        if (student == null)
        {
            TempData["ErrorMessage"] = "Student already deleted.";
            return RedirectToAction(nameof(Index));
        }

        // Remove from tracker
        _context.Students.Remove(student);
        await _context.SaveChangesAsync();

        TempData["SuccessMessage"] = "Student deleted successfully!";
    }
```

```
            catch (DbUpdateException ex)
            {
                _logger.LogError(ex, "Error deleting student {Id}", id);
                TempData["ErrorMessage"] =
                    "Cannot delete this student because they have related records. Remove enrollments first.";
            }

            return RedirectToAction(nameof(Index));
        }

        // Helper method to check existence
        private bool StudentExists(int id)
        {
            return _context.Students.Any(e => e.Id == id);
        }
    }
}
```

Key Components:

Async/Await — Non-blocking I/O

Every database operation is asynchronous, releasing the thread during I/O. await pauses the method without blocking the thread.

```
await _context.Students.ToListAsync();
```

→ See **Index()**, **Details()**, **Create()** POST, etc.

POST-REDIRECT-GET — PRG Pattern

After a successful form submission (POST), the server redirects to a GET action. This prevents duplicate submissions on page refresh.

```
return RedirectToAction(nameof(Index));
```

→ See **Create** POST, **Edit** POST, **DeleteConfirmed**.

ModelState Validation — Server-Side

Validates incoming data against model attributes ([Required], [EmailAddress]) and custom validation logic before writing to the database.

```
if (ModelState.IsValid) { ... }
```

→ Used in **Create** POST and **Edit** POST; also [Bind] prevents over-posting.

TempData — Cross-Request Messages

Stores short-lived data (typically success/error messages) that survives a redirect. Automatically cleared after being read.

```
TempData["SuccessMessage"] = "Student created!";
```

→ Shown in **Create**, **Edit**, **DeleteConfirmed**.

Anti-Forgery Tokens — CSRF Protection

Protects against Cross-Site Request Forgery attacks. The [ValidateAntiForgeryToken] attribute verifies a hidden token generated by the form.

```
[ValidateAntiForgeryToken]
```

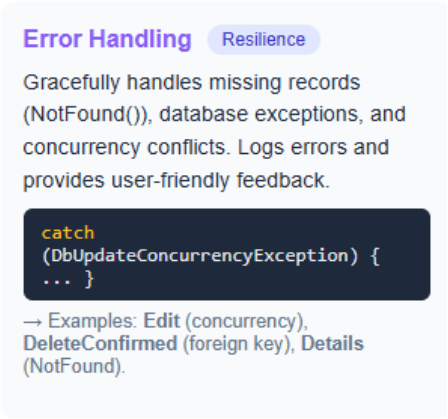
→ Applied to all state-changing POST actions (**Create**, **Edit**, **DeleteConfirmed**).

Explicit Route Attributes — Clean URLs

Define clean, RESTful URL patterns directly on actions using [HttpGet], [HttpPost] with templates. Improves readability and maintainability.

```
[HttpGet("Edit/{id}")]
```

→ See **Details**, **Edit**, **Delete** for parameterized routes.



Best Practice Considerations:

Use **AsNoTracking()** for read-only queries to improve performance

Always **validate anti-forgery tokens on state-changing POST requests**

Implement **proper error handling with user-friendly messages**

Use FindAsync() for primary key lookups - it checks the local cache first

Consider using **TryUpdateModelAsync** for more secure model binding in complex scenarios

The **"Async Trip"**: Always pair async in the method signature with **Task<IActionResult>** as the return type, and use await on the EF Core method.

Tracking vs. Saving: Methods like Add(), Update(), and Remove() only tell EF Core what you intend to do. The database is not actually touched until you call **SaveChangesAsync().**

Naming Convention: Asynchronous EF Core methods usually end in "Async" (e.g., FirstOrDefaultAsync, ToListAsync, AddAsync).

Note: In ASP.NET Core MVC, you will notice that we often explicitly label methods with [HttpPost], but we rarely see [HttpGet] or [HttpPut] used in standard Controller actions. Here is why:

- **The Default is Always GET**
  By convention, if you do not add an attribute (like [HttpPost]) to an action method, **ASP.NET Core assumes it is an HTTP GET request.** * **The Reason:** Most web interactions (loading a page, clicking a link, searching) are GET requests. To save developers time and keep code clean, the framework makes GET the "default" behaviour. If you want a method to be a GET, you simply don't have to say anything at all.
- **PUT vs. POST in MVC**

While **PUT** is a standard part of the HTTP protocol (used for updates), it is rarely used in standard "traditional" MVC Controllers:

- **HTML Form Limitations:** Standard HTML <form> tags only support two methods: GET and POST. Because browsers cannot natively send a "PUT" via a simple form submission, MVC developers use POST for both creating **and** updating records.
- **REST APIs vs. MVC:** You will see [HttpPut] used frequently in **Web APIs** (where JavaScript handles the request), but in a standard Controller returning a View(), [HttpPost] is the universal workhorse for sending data to the server.

3.3 Chapter Summary

In this chapter, we moved from the static structure of Data Models to the dynamic behavior of **Controllers**. We explored how the Controller acts as the "Middleman," orchestrating the flow of data between the user's browser and the database. We covered:

1. Controller Fundamentals

- **The Intermediary:** You learned that the Controller is responsible for accepting input, interacting with the Model, and selecting the appropriate View.
- **Base Class Power:** By inheriting from the Controller base class, we unlocked access to essential web helpers like View(), RedirectToAction(), and ModelState.
- **Dependency Injection (DI):** We moved away from manual object creation by "injecting" the ApplicationDbContext and ILogger into the constructor, following the Dependency Inversion Principle.

2. Action Methods & Results

- **Asynchronous Efficiency:** We implemented async Task<IActionResult> methods to ensure the server remains responsive while waiting for database I/O.
- **Flexible Responses:** You mastered returning various IActionResult types, such as View() for HTML, RedirectToAction() for workflow navigation, and NotFound() for error handling.
- **Security & UX:** We applied the **Post-Redirect-Get** pattern to prevent duplicate data submissions and used ModelState.IsValid to ensure data integrity.

3. EF Core in Controllers

- **CRUD Mastery:** You successfully implemented the full Create, Read, Update, and Delete cycle using _context methods like ToListAsync(), Add(), and Remove().

- **The Checkout Moment:** We distinguished between "tracking" changes and the final "checkout" via await _context.SaveChangesAsync(), which is the only moment SQL is sent to the DB.
- **Optimization:** We used FindAsync() for high-speed primary key lookups and Include() to prevent "N+1" query performance issues when loading related data.

Now that our Controller can fetch and save data, it's time to show that data to the user. In the next chapter, we will focus on **Views**, where we use **Razor Syntax** to turn C# data into beautiful, interactive HTML pages.

3.4 Review Questions

1. Why must every controller ASP.NET Core MVC inherit from the Controller base class, and what specific functionality does this provide to your action methods?
2. Explain the process of "Constructor Injection." Why is it better to inject the ApplicationDbContext rather than creating a new instance inside every method?
3. Explain the POST-REDIRECT-GET (PRG) pattern. Why is it considered a best practice after successful form submissions? What problem does it prevent? Provide a concrete example.
4. What is the primary benefit of using async Task<IActionResult> instead of a standard IActionResult when querying a database?
5. Describe a specific scenario where you would use RedirectToAction() instead of return View().
6. Compare synchronous (IActionResult) and asynchronous (Task<IActionResult>) action methods. Under what circumstances should you prefer async? What problem does async solve in web applications?
7. Why is it not strictly necessary to add the [HttpGet] attribute to a method that simply loads a page, but essential to add [HttpPost] to a method that saves data?
8. You are building a Library System. Write the constructor for a BookController that successfully injects both the ApplicationDbContext (to manage books) and an ILogger (to track errors).
9. Your e-commerce site has a ProductController. The Index action must display a list of products that belong to a specific category. Implement an asynchronous Index action that accepts an optional categoryId parameter. If categoryId is provided, filter products by that category; otherwise show all products. Use eager loading to include the Category navigation property. Return the appropriate view with the list of products.
10. You are developing a Student Information System. Write the Details action for a StudentController. It should accept an id parameter, retrieve the corresponding student from the database (including their Enrollments and Courses), and return a View. If the id is null or the student does not exist, return a 404 Not Found response. Log a warning when a student is not found.

4. Building the UI with Razor Views

Learning Outcomes:

By the end of this chapter, students should be able to:

- Master Razor Syntax & Integration – Use the @ symbol to transition seamlessly between HTML markup and C# code within a single file.
- Write conditional statements and loops (like @if and @foreach) to generate dynamic content based on data.
- Explain how the MVC engine uses naming conventions to link a Controller to its specific View folder.
- Organize .cshtml files correctly within the Views/[ControllerName] structure to ensure the framework can locate them.
- Scaffolding – Utilize Visual Studio's "MVC Controller with views, using Entity Framework" to automatically generate CRUD pages.
- Strongly-Typed Data – Declare a @model at the top of a view to enable IntelliSense and "type-safety" when accessing data sent from the Controller.
- Tag Helpers – Use specialized asp- attributes (like asp-for and asp-action) to generate clean HTML links and form elements.
- HTML Helpers – Implement standard helpers for complex UI tasks, ensuring your forms communicate correctly with your Action Methods.
- Tailoring Data for the UI – Use ViewModels to combine data from multiple entities in order to display aggregated information within a view

4.1 What is Razor?

Razor is a markup syntax that allows you to write HTML markup and C# code in the same .cshtml file, using the @ symbol to transition between the two worlds. It isn't a new programming language; rather, it is a server-side engine that "renders" your file into pure HTML before it ever reaches the user's browser. To protect your application, Razor automatically HTML-encodes output by default to prevent XSS (Cross-Site Scripting) attacks.

The Magic of the @ Symbol

The @ symbol is the "switch" that tells the Razor engine: *"Stop reading this as HTML and start reading it as C#"*.

Example: StudentProfile.cshtml

```
@* ========== MODEL DECLARATION ========== *@
<!-- Strongly-typed view: expects a Student object from the controller -->
@model SchoolApp.Models.Student

@* ========== INLINE EXPRESSIONS ========== *@
<h1>Welcome, @Model.FullName!</h1>
<p>Today is @DateTime.Now.ToString("dddd, MMMM d")</p>

@* ========== EXPLICIT EXPRESSION ========== *@
<p>Your age in dog years: @(Model.Age * 7)</p>
```

```
@* ========== CODE BLOCK ========== *@
@{
    var greeting = "Hello";
    var hour = DateTime.Now.Hour;
    if (hour < 12) greeting = "Good morning";
    else if (hour < 18) greeting = "Good afternoon";
    else greeting = "Good evening";
}

<h2>@greeting, @Model.FirstName!</h2>
```

```
@* ========== CONDITIONAL RENDERING ========== *@
@if (Model.IsEnrolled)
{
    <span class="badge bg-success">✅ Enrolled</span>
}
else
{
    <span class="badge bg-warning">⏳ Not enrolled</span>
}
```

```razor
@* ========== LOOPS ========== *@
<h3>Courses</h3>
@if (Model.Enrollments.Any())
{
    <ul>
    @foreach (var enrollment in Model.Enrollments)
    {
        <li>@enrollment.Course.Title (Grade: @enrollment.Grade)</li>
    }
    </ul>
}
else
{
    <p>No courses yet.</p>
}
```

```razor
@* ========== ESCAPING ========== *@
<p>Send feedback to @@support (this will render as @support)</p>

@* ========== RAW HTML (use with CAUTION) ========== *@
<div>
    @Html.Raw(Model.Biography) <!-- Assumes Biography contains trusted HTML -->
</div>
```

The @ Symbol – Your Bridge

Model declaration: @model Student – tells Razor what type of object to expect from the controller.

Inline expression: @DateTime.Now – writes the value directly to the HTML output.

Explicit expression: @(1 + 2) – use parentheses when the expression contains spaces or operators.

Code block: @{ ... } – execute multiple C# statements without writing to output.

Control structures: @if, @foreach, @switch – mix logic and markup seamlessly.

Escaping: @@ – to display a literal @ symbol.

Comments: @* ... *@ – server-side comments (not sent to the client).

Strongly-Typed Views

The @model directive at the top of a .cshtml file declares the exact type of the model object that the controller will pass to the view. This enables:

- **IntelliSense** – full code completion for model properties.
- **Type safety** – compile-time checking; you cannot accidentally pass the wrong object.
- **Clarity** – any developer reading the view knows exactly what data to expect.

4.2 Project Organization & Naming Conventions

ASP.NET Core MVC uses a powerful set of conventions to automatically map controllers, actions, and views without requiring explicit configuration. Understanding this hierarchy is essential for organizing large applications.

4.2.1 The Default View Discovery Path

The engine searches in this exact order. The first match wins.

1. /Views/{ControllerName}/{ActionName}.cshtml
2. /Views/Shared/{ActionName}.cshtml
3. /Pages/Shared/{ActionName}.cshtml (Razor Pages)
4. /Views/Shared/DisplayTemplates/{ActionName}.cshtml (if applicable)

> **Naming Rules & Conventions**
>
> **Controller class:** Must end with Controller (e.g., ProductController).
>
> **Controller folder:** Typically placed in Controllers/ (optional but recommended).
>
> **Views folder:** Must be named exactly as the controller without the "Controller" suffix.
>
> **Action methods:** Map to .cshtml files with the same name inside the controller's view folder.
>
> **Shared views:** Reside in /Views/Shared/ — accessible from any controller.
>
> **Partial views:** Conventionally prefixed with underscore (_LoginPartial.cshtml) but not required.
>
> **Layout files:** Typically _Layout.cshtml in /Views/Shared/ or /Pages/.

🎯 **The Golden Rule:** If your controller is named `StudentController`, its views **must** live in `/Views/Student/`. When you call `return View();` inside an action named `Index`, MVC automatically looks for `/Views/Student/Index.cshtml`.

🗁 Typical MVC Project Structure

```
📁 Controllers/
   ├── HomeController.cs
   ├── ProductController.cs
   └── StudentController.cs
📁 Models/
   ├──Student.cs
   └──Product.cs

📁 Views/
   ├── Home/
   │    ├── Index.cshtml
   │    └── Privacy.cshtml
   ├── Product/
   │    ├── Index.cshtml
   │    └── Details.cshtml
   ├── Student/
   │    ├── Index.cshtml
   │    ├── Create.cshtml
   │    └── Edit.cshtml

   ├── Shared/
   │    ├── _Layout.cshtml
   │    ├── _ValidationScriptsPartial.cshtml
   │    ├── Error.cshtml
   │    └── _LoginPartial.cshtml
   ├── _ViewStart.cshtml      ← Runs before every view (sets Layout)
   └── _ViewImports.cshtml    ← Global using directives, tag helpers
```

Controller-to-View Mapping Examples

```
// Inside StudentController.cs
public class StudentController : Controller
{
    public IActionResult Index()
    {
        // Looks for /Views/Student/Index.cshtml
        return View();
    }

    public IActionResult Details(int id)
    {
        // Explicitly specify a view name
        return View("Profile", model); // Looks for /Views/Student/Profile.cshtml
    }

    public IActionResult About()
    {
        // Absolute path - overrides conventions
        return View("~/Views/Home/About.cshtml");
    }
}
```

⚠ **Important:** When you use return View("SomeView"), MVC first searches in the controller-specific folder (/Views/Student/SomeView.cshtml), then falls back to /Views/Shared/SomeView.cshtml. Use absolute paths (~/Views/...) to bypass this search order.

4.3 Scaffolding Controllers with Views using EF Core

Scaffolding is a code generation feature in Visual Studio that instantly creates a fully functional controller and corresponding Razor views for a given model class and database context. It saves hours of repetitive coding and ensures best practices are followed out of the box.

🛠️ How to Scaffold (Visual Studio)

1. **Right-click** the `Controllers` folder → **Add** → **New Scaffolded Item**.
2. Select **MVC Controller with views, using Entity Framework**.
3. Choose your **Model class** (e.g., `Student`) and **Data context class** (e.g., `ApplicationDbContext`).
4. Check **Generate views** (always) and optionally **Reference script libraries** and **Use a layout page**.
5. Click **Add**. Visual Studio generates a controller and five Razor views in seconds.

What Gets Generated?

- **Controller** – StudentController.cs with full CRUD actions (Index, Details, Create, Edit, Delete).
- **Views folder** – /Views/Student/ with five .cshtml files:
    - Index.cshtml – lists all records
    - Details.cshtml – shows single record
    - Create.cshtml – form to add new record
    - Edit.cshtml – form to edit existing record
    - Delete.cshtml – confirmation page
- **All code is asynchronous** – uses async Task<IActionResult> and SaveChangesAsync().
- **Tag Helpers** – forms use asp-for and asp-action.
- **Validation** – ModelState.IsValid and validation summaries are included.
- **Anti-forgery tokens** – [ValidateAntiForgeryToken] on POST actions.

Generated Project Structure:

```
Controllers/
    └── StudentController.cs
Models/
    └── Student.cs
Views/
    └── Student/
        ├── Index.cshtml
        ├── Details.cshtml
        ├── Create.cshtml
        ├── Edit.cshtml
        └── Delete.cshtml
```

> ⚠ **Important:** Scaffolding is a **one-time generation**. Once you modify the generated files, re-scaffolding will overwrite your changes. Always back up custom logic or use partials if you anticipate needing to regenerate. For production applications, treat scaffolded code as a **starting point**, not the final product.

Generated Controller:

```
public class StudentController : Controller
{
    private readonly ApplicationDbContext _context;

    public StudentController(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET: Student
    public async Task<IActionResult> Index()
    {
        return View(await _context.Students.ToListAsync());
    }
```

```
    // GET: Student/Details/5
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null) return NotFound();

        var student = await _context.Students
            .FirstOrDefaultAsync(m => m.Id == id);

        if (student == null) return NotFound();

        return View(student);
    }
```

// Additional CRUD actions (Create POST, Edit, Delete) omitted for brevity...

Generated View: Index.cshtml

```
@model IEnumerable<SchoolApp.Models.Student>

@{
    ViewData["Title"] = "Index";
}

<h1>Students</h1>

<p>
    <a asp-action="Create">Create New</a>
</p>
```

```
<table class="table">
    <thead>
        <tr>
            <th>@Html.DisplayNameFor(model => model.Name)</th>
            <th>@Html.DisplayNameFor(model => model.Email)</th>
            <th></th>
        </tr>
    </thead>
```

```
    <tbody>
@foreach (var item in Model) {
        <tr>
            <td>@Html.DisplayFor(modelItem => item.Name)</td>
            <td>@Html.DisplayFor(modelItem => item.Email)</td>
            <td>
                <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
                <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
                <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
            </td>
        </tr>
}
    </tbody>
</table>
```

Student Tip: Use scaffolding to **learn** best practices and **bootstrap** your projects. The generated code demonstrates proper async patterns, dependency injection, tag helpers, and the POST-REDIRECT-GET pattern. Study it, then customize it to fit your specific business logic.

4.4 Strongly-Typed Views – How the View "Receives" the Model

A **strongly-typed view** is a Razor view that declares the exact type of the model object it expects from the controller using the @model directive. This creates a contract between the controller and the view, enabling compile-time checking and full IntelliSense support.

🎯 The Contract

- **Controller:** `return View(student);` – passes a `Student` object to the view.
- **View:** `@model Student` – promises to accept a `Student` object.
- **Result:** If the controller passes the wrong type (or forgets to pass one), the compiler raises an error.

Benefits of Strongly-Typed Views:

IntelliSense: Type @Model. and get a dropdown of all available properties and methods.
Compile-time checking: A missing or mismatched model type breaks the build — no runtime surprises.
Self-documenting: The @model declaration tells other developers exactly what data the view requires.

Refactoring support: Renaming a model property automatically updates all references across views.

Controller – Passing the Model (via an Action Method)

```
// GET: Student/Details/5
public async Task<IActionResult> Details(int? id)
{
    if (id == null) return NotFound();

    // Retrieve a single Student from the database
    var student = await _context.Students
        .FirstOrDefaultAsync(s => s.Id == id);

    if (student == null) return NotFound();

    // Pass the Student object to the view
    return View(student);  // ← Strongly-typed model
}

// GET: Student/Index
public async Task<IActionResult> Index()
{
    // Pass a list of students
    var students = await _context.Students.ToListAsync();
    return View(students);  // ← Model is IEnumerable<Student>
}
```

View: Details.cshtml (Strongly-Typed)

```
@* Declare the model type – this view expects a single Student object *@
@model SchoolApp.Models.Student

@{
    ViewData["Title"] = "Student Details";
}

<h1>Student Details</h1>

<div>
    <h4>Student</h4>
    <hr />
```

```
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Email)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Email)
        </dd>
```

```
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.EnrollmentDate)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.EnrollmentDate)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model.Id">Edit</a> |
    <a asp-action="Index">Back to List</a>
</div>
```

View: Index.cshtml (List of Students)

```
@* This view expects a collection of Student objects *@
@model IEnumerable<SchoolApp.Models.Student>

@{
    ViewData["Title"] = "Student List";
}

<h1>All Students</h1>

<p>
    <a asp-action="Create">Create New</a>
</p>

@if (!Model.Any())
{
    <p>No students found.</p>
}
```

```
else
{
    <table class="table">
        <!-- ... table header ... -->
        @foreach (var item in Model)
        {
            <tr>
                <td>@item.Name</td>
                <td>@item.Email</td>
                <td>
                    <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
                    <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
                    <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
                </td>
            </tr>
        }
    </table>
}
```

💡**Strongly-Typed Views in Action**: The @model directive creates a variable called Model (capital M) that holds the passed object.
Use @Model.PropertyName anywhere in the view. Visual Studio provides IntelliSense as soon as you type @Model. – a huge productivity boost.

4.5 ViewData, ViewBag & TempData – Passing Auxiliary Data

While the @model directive is used for the primary data entity, you often need to pass **auxiliary data** to the view – dropdown lists, page titles, success messages, etc. ASP.NET Core MVC provides three containers for this purpose: ViewData, ViewBag, and TempData.

Which One to Use?

Container	Life-time Access	Syntax	Use Case
ViewData	Single request	@ViewData[“Key”]	Dictionary style, requires casting
ViewBag	Single request	@ViewBag.Key	Dictionary style, requires casting

TempData	Across Redirects	@TempData[“Key”]	Pass messages after POST-REDIRECT-GET

✅ **Most common practice:** Use ViewBag for dropdown lists, page titles, and other simple view data because it requires no casting and has cleaner syntax.

4.5.1 Creating a SelectList

A SelectList is a specialized collection used to populate HTML <select> dropdowns. It holds value/text pairs and optionally tracks a selected value. You construct it from any data source (List, Array, Database query) and pass it via ViewBag.

Example:

ViewBag.CategoryList = new SelectList(categories, "Id", "Name");

Let’s look at an example of how a controller passes a dropdown list via the ViewBag container and how a view consumes the dropdown list.

Controller – Passing Dropdown via ViewBag

```
public class ProductController : Controller
{
    private readonly ApplicationDbContext _context;

    public ProductController(ApplicationDbContext context)
    {
        _context = context;
    }
```

```
    // GET: Product/Create
    public IActionResult Create()
    {
        // 1. Retrieve categories from database
        var categories = _context.Categories.ToList();

        // 2. Create SelectList: data, value field, text field
        var selectList = new SelectList(categories, "Id", "Name");

        // 3. Pass via ViewBag (most common – no casting needed in view)
        ViewBag.CategoryList = selectList;

        // 4. (Optional) You can also use ViewData – requires casting
        ViewData["CategoryList"] = selectList;

        // 5. Strongly-typed model for the form (if any)
        return View();
    }
```

```
    // GET: Product/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null) return NotFound();

        // Pass categories with the product's current category pre-selected
        var categories = _context.Categories.ToList();
        ViewBag.CategoryList = new SelectList(categories, "Id", "Name", product.CategoryId);

        return View(product);
    }
}
```

View: Create.cshtml – Consuming the Dropdown

```
@model Product

@{
    ViewData["Title"] = "Create Product";
}

<h1>Create Product</h1>

<form asp-action="Create">
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
    </div>
```

```
    <div class="form-group">
        <label asp-for="CategoryId"></label>

        <!-- USING VIEWBAG (most common) - no casting needed -->
        <select asp-for="CategoryId"
                asp-items="ViewBag.CategoryList"
                class="form-control">
            <option value="">-- Select Category --</option>
        </select>

        <!-- Alternative using ViewData (requires casting) -->
        @* <select asp-for="CategoryId" asp-items="@(ViewData["CategoryList"] as SelectList)"> ... </select> *@
    </div>

    <button type="submit" class="btn btn-primary">Create</button>
</form>
```

4.5.2 TempData for User Feedback

TempData survives a single redirect, making it perfect for the **POST-REDIRECT-GET** pattern. Store a success or error message in the POST action, then display it in the redirected GET view. The message is automatically cleared after being read.

> ⚠ **Important:** ViewBag and ViewData are **not** type-safe. Misspelled keys or incorrect casting will only fail at runtime. Use strongly-typed ViewModels for complex data; reserve these containers for simple, view-specific auxiliary data.

TempData – POST-REDIRECT-GET Messages

```
// POST: Product/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
```

```
        // Store success message in TempData
        TempData["SuccessMessage"] = $"Product '{product.Name}' created successfully!";

        return RedirectToAction(nameof(Index));
    }

    // If we got this far, something failed; repopulate dropdown
    var categories = _context.Categories.ToList();
    ViewBag.CategoryList = new SelectList(categories, "Id", "Name", product.CategoryId);
    return View(product);
}
```

```
// GET: Product/Index
public async Task<IActionResult> Index()
{
    var products = await _context.Products.Include(p => p.Category).ToListAsync();
    return View(products);
}
```

View: Index.cshtml – Displaying TempData

```
@model IEnumerable<Product>

@{
    ViewData["Title"] = "Products";
}

<!-- Display success message from TempData -->
@if (TempData["SuccessMessage"] != null)
{
    <div class="alert alert-success alert-dismissible fade show" role="alert">
        @TempData["SuccessMessage"]
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
}
```

```
<h1>Products</h1>
<p>
    <a asp-action="Create">Create New</a>
</p>

<!-- ... table of products ... -->
```

📌 Summary:

ViewBag is the preferred container for dropdown lists and simple view data – clean syntax, no casting.

SelectList constructor: new SelectList(collection, "valueField", "textField", selectedValue).

Use asp-items="ViewBag.CategoryList" in the <select> tag helper.

TempData is essential for POST-REDIRECT-GET success/error messages.

Always consider ViewModels for complex data; ViewBag/ViewData are for auxiliary, presentation-only data.

4.6 Razor Views with JS and CSS

Razor views are HTML-first, but they can seamlessly include JavaScript and CSS to create rich, interactive user experiences. This hands-on project will guide you through creating a complete product catalog with dynamic filtering using Razor, CSS, and JavaScript.

📁 1. Project Creation & Setup

1. Open Visual Studio and select **Create a new project → ASP.NET Core Web App (Model-View-Controller)**.
2. Name the project `ProductCatalog`, choose a location, and click **Create**.
3. Target .NET 8.0 (or later) with default settings.
4. Open **Tools → NuGet Package Manager → Manage NuGet Packages for Solution** and install:
   - `Microsoft.EntityFrameworkCore.SqlServer`
   - `Microsoft.EntityFrameworkCore.Tools`
   - `Microsoft.EntityFrameworkCore.Design`

2. Models & Database Context

- In the Models folder, add Category.cs and Product.cs
- Add the [Column(TypeName = "decimal(18,2)")] attribute to the Price property to ensure correct SQL Server decimal precision.

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Navigation property
    public ICollection<Product> Products { get; set; }
}
```

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }

    [Column(TypeName = "decimal(18,2)")] // Ensures proper decimal precision in SQL Server
    public decimal Price { get; set; }

    public string Description { get; set; }
    public string ImageUrl { get; set; }

    // Foreign key
    public int CategoryId { get; set; }

    // Navigation property
    public Category Category { get; set; }
}
```

- Create a Data folder. Add ApplicationDbContext.cs .
- In appsettings.json, add a connection string for LocalDB.
- In Program.cs, register ApplicationDbContext with UseSqlServer.
- Seed the database with categories and products inside OnModelCreating.

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Category> Categories { get; set; }
    public DbSet<Product> Products { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Seed Categories
        modelBuilder.Entity<Category>().HasData(
            new Category { Id = 1, Name = "Electronics" },
            new Category { Id = 2, Name = "Home & Kitchen" },
            new Category { Id = 3, Name = "Audio" },
            new Category { Id = 4, Name = "Sports & Outdoors" }
        );
```

```
    // Seed Products
    modelBuilder.Entity<Product>().HasData(
        new Product { Id = 1, Name = "Laptop", Price = 12999.99m,
                      Description = "High-performance laptop for work and play",
                      ImageUrl = "/images/laptop.jpg", CategoryId = 1 },
        new Product { Id = 2, Name = "Wireless Mouse", Price = 349.99m,
                      Description = "Ergonomic wireless mouse",
                      ImageUrl = "/images/mouse.jpg", CategoryId = 1 },
        new Product { Id = 3, Name = "Mechanical Keyboard", Price = 899.99m,
                      Description = "RGB mechanical keyboard with blue switches",
                      ImageUrl = "/images/keyboard.jpg", CategoryId = 1 },
        new Product { Id = 4, Name = "4K Monitor", Price = 4999.99m,
                      Description = "27-inch 4K UHD monitor",
                      ImageUrl = "/images/monitor.jpg", CategoryId = 1 },
```

```
        new Product { Id = 5, Name = "Noise-Cancelling Headphones", Price = 2499.99m,
                      Description = "Wireless over-ear headphones",
                      ImageUrl = "/images/headphones.jpg", CategoryId = 3 },
        new Product { Id = 6, Name = "Coffee Maker", Price = 1299.99m,
                      Description = "Programmable coffee machine",
                      ImageUrl = "/images/coffee.jpg", CategoryId = 2 },
        new Product { Id = 7, Name = "Bluetooth Speaker", Price = 799.99m,
                      Description = "Portable waterproof speaker",
                      ImageUrl = "/images/speaker.jpg", CategoryId = 3 },
        new Product { Id = 8, Name = "Yoga Mat", Price = 449.99m,
                      Description = "Non-slip exercise mat",
                      ImageUrl = "/images/yogamat.jpg", CategoryId = 4 }
    );
}
```

(appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=
(localdb)\\mssqllocaldb;Database=ProductCatalogDB;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

(Program.cs)

```
using Microsoft.EntityFrameworkCore;
using ProductCatalog.Data;

var builder = WebApplication.CreateBuilder(args);
// Register ApplicationDbContext with SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllersWithViews();

var app = builder.Build();
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

```
app.UseHttpsRedirection();
// Enable serving static files from wwwroot (required for images, CSS, JS)
app.UseStaticFiles();

app.UseRouting();
app.UseAuthorization();

app.MapStaticAssets();

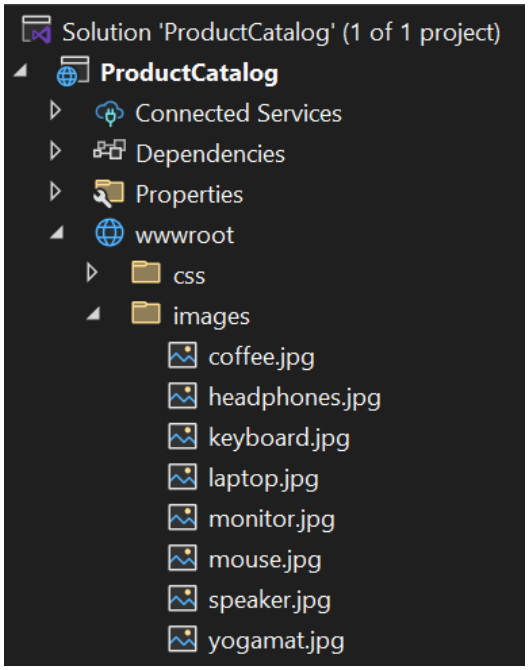
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

- Open Package Manager Console and run:
    - Add-Migration InitialCreate
    - Update-Database

3. wwwroot/images – Product Images

- In Solution Explorer, right-click wwwroot → Add → New Folder and name it images.
- Download or create placeholder product images (e.g., laptop.jpg, mouse.jpg) and place them inside the wwwroot/images folder.
- If you don't have images, the view includes a fallback to /images/placeholder.jpg. You can also comment out the <img> tag or use a Font Awesome icon instead.
- Ensure app.UseStaticFiles() is present in Program.cs to serve static files.



4. Controller & Actions

- Create ProductController with constructor injecting ApplicationDbContext.
- Add an asynchronous Index action that:
    - Retrieves all products (include Category).
    - Retrieves all categories, creates a SelectList (value = Id, text = Name).
    - Stores the SelectList in ViewBag.CategoryList.
    - Passes the products to the view.

```
public class ProductController : Controller
{
    private readonly ApplicationDbContext _context;

    public ProductController(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        // 1. Get products with Category (eager loading)
        var products = await _context.Products
            .Include(p => p.Category)
            .ToListAsync();
```

```
        // 2. Get all categories for the dropdown
        var categories = await _context.Categories.ToListAsync();

        // 3. Create SelectList (value = Id, text = Name, optional selected value)
        ViewBag.CategoryList = new SelectList(categories, "Id", "Name");

        // 4. Pass products to view
        return View(products);
    }
}
```

5. Navigation – Link to Product Catalog

- Open Views/Shared/_Layout.cshtml.
- Locate the navbar <ul class="navbar-nav"> section.
- Add a new <li> item with an anchor tag linking to the Product Index:

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Product" asp-action="Index">Products</a>
</li>
```

- This adds a "Products" link in the navigation bar that takes users directly to your product catalog.

6. Razor View & Client-Side Filtering

- Create Views/Product/Index.cshtml with @model IEnumerable<Product>.
- Add a <style> block to define responsive product cards and layout.
- Add a search input and a category dropdown (using ViewBag.CategoryList).
- Wrap the product cards in a container with data-* attributes for name, description, and category.
- Add a <script> block that listens to both input and dropdown changes and filters cards accordingly.
- Display the price in South African Rand (R) using R@product.Price.ToString("F2").

```
@model IEnumerable<Product>

@{
    ViewData["Title"] = "Product Catalog";
}

<style>
    .product-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
        gap: 1.5rem;
        margin-top: 2rem;
    }

    .product-card {
        border: 1px solid #e2e8f0;
        border-radius: 12px;
        padding: 1.2rem;
        background: white;
        box-shadow: 0 4px 6px rgba(0,0,0,0.05);
        transition: transform 0.2s, box-shadow 0.2s;
    }

        .product-card:hover {
            transform: translateY(-5px);
            box-shadow: 0 10px 20px rgba(0,0,0,0.1);
        }

    .product-image {
        width: 100%;
        height: 160px;
        object-fit: cover;
        border-radius: 8px;
        margin-bottom: 1rem;
    }
```

```
    .product-title {
        font-size: 1.2rem;
        font-weight: 600;
        margin-bottom: 0.4rem;
    }

    .product-price {
        font-size: 1.4rem;
        font-weight: 700;
        color: #3b82f6;
        margin-bottom: 0.4rem;
    }

    .product-description {
        color: #64748b;
        margin-bottom: 1rem;
        font-size: 0.9rem;
    }

    .product-category {
        font-size: 0.8rem;
        color: #6b7280;
        margin-bottom: 0.5rem;
    }

    .filter-bar {
        display: flex;
        gap: 1rem;
        margin-bottom: 1.5rem;
        flex-wrap: wrap;
    }
```

```
    .search-box {
        flex: 2;
        min-width: 200px;
        padding: 0.75rem 1rem;
        font-size: 1rem;
        border: 2px solid #e2e8f0;
        border-radius: 8px;
        transition: border-color 0.2s;
    }

        .search-box:focus {
            outline: none;
            border-color: #3b82f6;
        }

    .category-dropdown {
        flex: 1;
        min-width: 150px;
        padding: 0.75rem 1rem;
        font-size: 1rem;
        border: 2px solid #e2e8f0;
        border-radius: 8px;
        background: white;
    }

    .no-products {
        text-align: center;
        padding: 2rem;
        color: #64748b;
        font-size: 1.1rem;
        grid-column: 1 / -1;
    }
</style>
```

```
<h1>Our Products</h1>

<!-- Filter bar: Search + Category Dropdown -->
<div class="filter-bar">
    <input type="text" id="searchInput" class="search-box" placeholder="🔍 Search by name or description..." />

    <select id="categoryFilter" class="category-dropdown">
        <option value="">All Categories</option>
        @foreach (var category in ViewBag.CategoryList)
        {
            <option value="@category.Value">@category.Text</option>
        }
    </select>
</div>

<!-- Product grid -->
<div id="productContainer" class="product-grid">
    @foreach (var product in Model)
    {
        <div class="product-card"
             data-name="@product.Name.ToLower()"
             data-description="@product.Description.ToLower()"
             data-category="@product.CategoryId">
            <img src="@(string.IsNullOrEmpty(product.ImageUrl) ? "/images/placeholder.jpg" : product.ImageUrl)"
                 alt="@product.Name" class="product-image" />
            <div class="product-title">@product.Name</div>
            <div class="product-price">R @product.Price.ToString("F2")</div>
            <div class="product-category">@product.Category?.Name</div>
            <div class="product-description">@product.Description</div>
            <a href="#" class="btn btn-primary">View Details</a>
        </div>
    }
</div>
```

```
<script>
    document.addEventListener('DOMContentLoaded', function() {
        const searchInput = document.getElementById('searchInput');
        const categoryFilter = document.getElementById('categoryFilter');
        const productCards = Array.from(document.querySelectorAll('.product-card'));

        function filterProducts() {
            const searchTerm = searchInput.value.toLowerCase().trim();
            const selectedCategory = categoryFilter.value;

            let visibleCount = 0;
            productCards.forEach(card => {
                const name = card.dataset.name;
                const description = card.dataset.description;
                const category = card.dataset.category;

                const matchesSearch = searchTerm === '' ||
                    name.includes(searchTerm) ||
                    description.includes(searchTerm);
                const matchesCategory = selectedCategory === '' ||
                    category === selectedCategory;

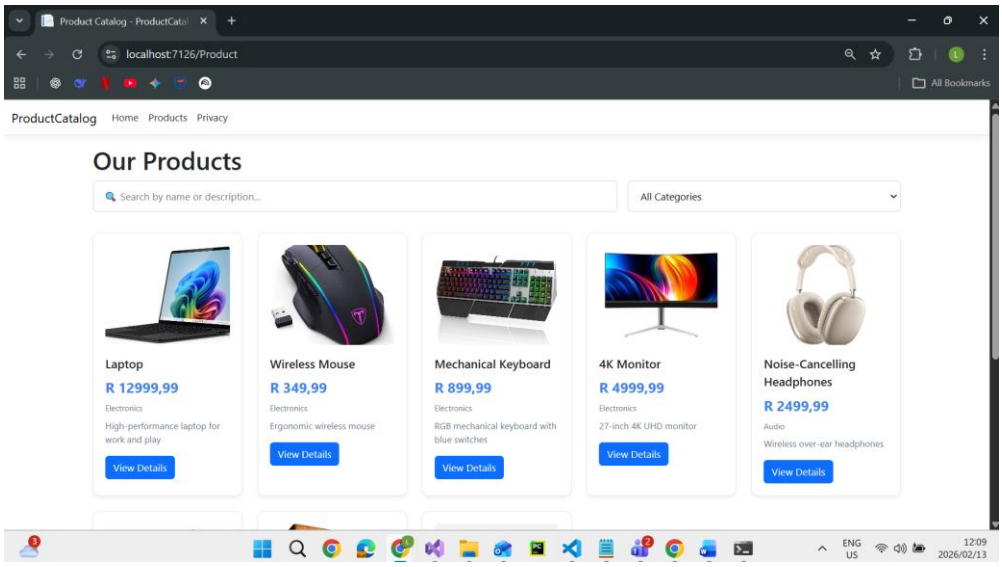
                if (matchesSearch && matchesCategory) {
                    card.style.display = '';
                    visibleCount++;
                } else {
                    card.style.display = 'none';
                }
            });
```

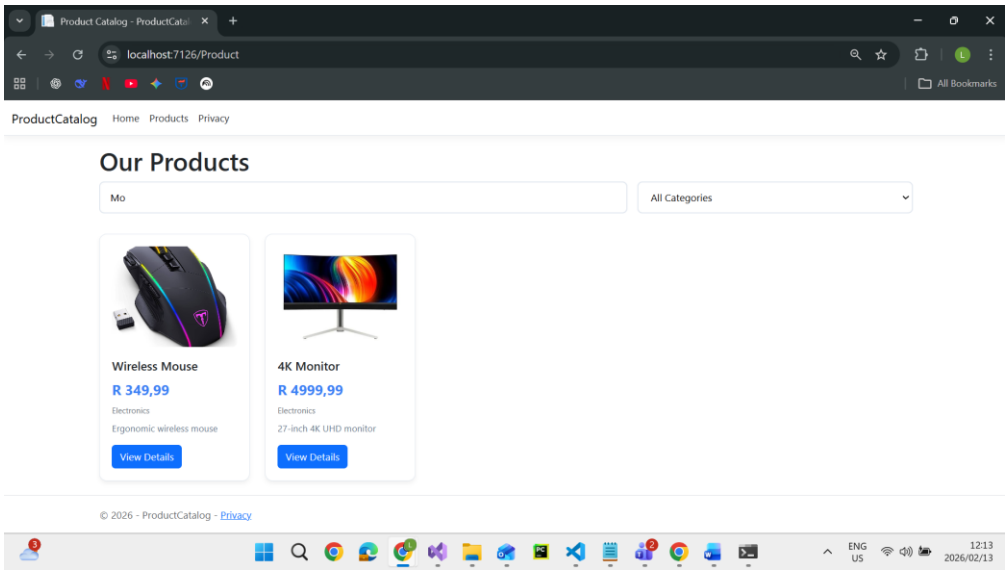
```
            const container = document.getElementById('productContainer');
            let noResults = document.getElementById('noResultsMsg');
            if (visibleCount === 0) {
                if (!noResults) {
                    noResults = document.createElement('div');
                    noResults.id = 'noResultsMsg';
                    noResults.className = 'no-products';
                    noResults.textContent = 'No products match your filters.';
                    container.appendChild(noResults);
                }
            } else {
                if (noResults) noResults.remove();
            }
        }

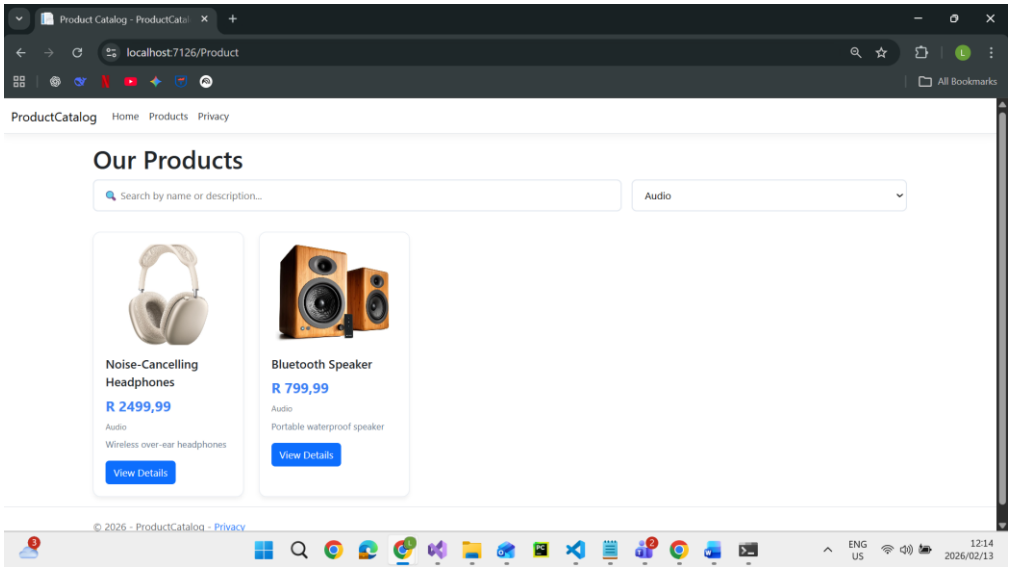
        searchInput.addEventListener('input', filterProducts);
        categoryFilter.addEventListener('change', filterProducts);
    });
</script>
```

✅ **By the end, you will have:** A fully functional product catalog with live filtering by text search and category, styled with modern CSS, seeded with products, and accessible via a dedicated navigation link.

Output:


Product Catalog - ProductCata
localhost:7126/Product
All Bookmarks
ProductCatalog
Home
Products
Privacy
Our Products
Search by name or description...
All Categories
Laptop
R 12999,99
Electronics
High-performance laptop for work and play
View Details
Wireless Mouse
R 349,99
Electronics
Ergonomic wireless mouse
View Details
Mechanical Keyboard
R 899,99
Electronics
RGB mechanical keyboard with blue switches
View Details
4K Monitor
R 4999,99
Electronics
27-inch 4K UHD monitor
View Details
Noise-Cancelling Headphones
R 2499,99
Audio
Wireless over-ear headphones
View Details
ENG
US
12:09
2026/02/13


Product Catalog - ProductCata
localhost:7126/Product
All Bookmarks
ProductCatalog
Home
Products
Privacy
Our Products
Mo
All Categories
Wireless Mouse
R 349,99
Electronics
Ergonomic wireless mouse
View Details
4K Monitor
R 4999,99
Electronics
27-inch 4K UHD monitor
View Details
© 2026 - ProductCatalog - Privacy
ENG
US
12:13
2026/02/13


Product Catalog - ProductCata
localhost:7126/Product
All Bookmarks
ProductCatalog
Home
Products
Privacy
Our Products
Search by name or description...
Audio
Noise-Cancelling Headphones
R 2499,99
Audio
Wireless over-ear headphones
View Details
Bluetooth Speaker
R 799,99
Audio
Portable waterproof speaker
View Details
© 2026 - ProductCatalog - Privacy
ENG
US
12:14
2026/02/13

4.7 HTML Helpers & Tag Helpers

ASP.NET Core MVC provides two powerful ways to generate HTML elements while keeping your views dynamic and maintainable: **HTML Helpers** (C# methods) and **Tag Helpers** (HTML-friendly attributes). Both reduce repetitive markup and automatically handle things like form fields, validation, and URL generation.

HTML Helpers

- C# methods that return HTML strings, called with @Html..
- Examples: @Html.TextBoxFor(), @Html.ActionLink(), @Html.BeginForm().
- Strongly-typed versions (...For) use lambda expressions to bind to model properties.
- **Pros:** Familiar to developers from older ASP.NET MVC; IntelliSense available.
- **Cons:** HTML is hidden inside C# code; can feel less natural for designers.

Tag Helpers

- Special attributes that attach to existing HTML elements, transforming them on the server.
- Examples: <input asp-for="Name" />, <a asp-action="Index">.
- Look and feel like standard HTML – designers can work with them easily.
- **Pros:** Clean, readable markup; full IntelliSense (with @addTagHelper); built-in support for many scenarios (forms, links, caching, environment).
- **Cons:** Requires enabling in _ViewImports.cshtml (though enabled by default in new projects).

> **Recommendation:** Use **Tag Helpers** for new development. They produce cleaner markup and integrate seamlessly with modern front-end workflows. HTML Helpers are still supported and useful when you need to generate HTML from C# logic.

Below is an example of how a form would look like when you're using Tag Helpers vs when you're using HTML Helpers:

Tag Helpers

```
@model Product

<form asp-action="Create" method="post" class="mt-3">
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label asp-for="Price"></label>
        <input asp-for="Price" class="form-control" />
        <span asp-validation-for="Price" class="text-danger"></span>
    </div>

    <div class="form-group">
        <label asp-for="CategoryId"></label>
        <select asp-for="CategoryId" asp-items="ViewBag.CategoryList" class="form-control">
            <option value="">-- Select Category --</option>
        </select>
        <span asp-validation-for="CategoryId" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Create</button>
    <a asp-action="Index" class="btn btn-secondary">Back to List</a>
</form>
```

HTML Helpers

```
@model Product

@using (Html.BeginForm("Create", "Product", FormMethod.Post, new { @class = "mt-3" }))
{
    <div class="form-group">
        @Html.LabelFor(m => m.Name)
        @Html.TextBoxFor(m => m.Name, new { @class = "form-control" })
        @Html.ValidationMessageFor(m => m.Name, "", new { @class = "text-danger" })
    </div>

    <div class="form-group">
        @Html.LabelFor(m => m.Price)
        @Html.TextBoxFor(m => m.Price, new { @class = "form-control" })
        @Html.ValidationMessageFor(m => m.Price, "", new { @class = "text-danger" })
    </div>

    <div class="form-group">
        @Html.LabelFor(m => m.CategoryId)
        @Html.DropDownListFor(m => m.CategoryId, (SelectList)ViewBag.CategoryList, "-- Select --", new { @class = "form-control" })
        @Html.ValidationMessageFor(m => m.CategoryId, "", new { @class = "text-danger" })
    </div>

    <button type="submit" class="btn btn-primary">Create</button>
    @Html.ActionLink("Back to List", "Index", "Product", null, new { @class = "btn btn-secondary" })
}
```

4.8 Tailoring Data for the UI using ViewModels

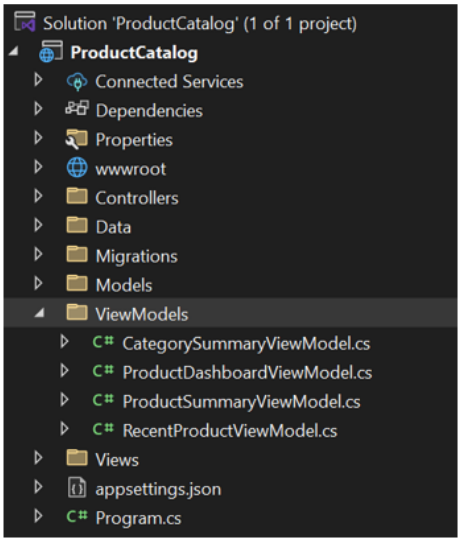
A **ViewModel** is a simple, non-persistent C# class designed specifically to meet the data requirements of a single view. Unlike domain models (Entities), ViewModels are not mapped to database tables—they exist solely to shape and combine data for presentation.

🎯 Why ViewModels?

- **Combine multiple entities** – e.g., a dashboard showing product totals, categories, and recent additions.
- **Shape data** – flatten complex object graphs, select only the properties needed by the view.
- **Avoid over-posting** – use ViewModels for Create/Edit to expose only safe properties.
- **Add view-specific metadata** – e.g., dropdown lists, formatted prices, calculated fields.
- **Decouple UI from domain** – changes to the database schema don't force view changes, and vice versa.

🗀 Where to put ViewModels?

Create a ViewModels folder at the project root. Name them after the view they serve (e.g., ProductDashboardViewModel). This keeps your project organized and discoverable.



💡 **Best Practice:** In a real-world application, avoid passing domain entities directly to views – always use ViewModels for complex dashboards or forms. This decouples your UI from your data layer and gives you full control over what data is exposed.

In our Product Catalog web application from the previous section, we may need to display aggregated information that spans multiple products and categories. Let's continue with the

same project to demonstrate how we can use ViewModels to combine data from multiple entities (domain models)

```
/// <summary>
/// ViewModel for the product dashboard - combines multiple data sources
/// </summary>
public class ProductDashboardViewModel
{
    // Summary statistics
    public int TotalProducts { get; set; }
    public int TotalCategories { get; set; }
    public decimal AveragePrice { get; set; }

    // List of recently added products
    public List<RecentProductViewModel> RecentProducts { get; set; }

    // Top most expensive products
    public List<ProductSummaryViewModel> TopExpensiveProducts { get; set; }

    // Categories with product counts
    public List<CategorySummaryViewModel> CategoriesWithCount { get; set; }
}
```

```
// Nested ViewModels - each tailored to a specific display need
public class RecentProductViewModel
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string CategoryName { get; set; }
    public DateTime DateAdded { get; set; } // We'll use Id as proxy since we don't have a date
}

public class ProductSummaryViewModel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string CategoryName { get; set; }
}

public class CategorySummaryViewModel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int ProductCount { get; set; }
}
```

```
1 reference
public class DashboardController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public DashboardController(ApplicationDbContext context)
    {
        _context = context;
    }

    0 references
    public async Task<IActionResult> Index()
    {
        // 1. Instantiate the ViewModel
        var viewModel = new ProductDashboardViewModel();

        // 2. Summary statistics
        viewModel.TotalProducts = await _context.Products.CountAsync();
        viewModel.TotalCategories = await _context.Categories.CountAsync();
        viewModel.AveragePrice = await _context.Products.AverageAsync(p => p.Price);

        // 3. Recent products (by Id as a proxy for "newest" - in real app use DateAdded)
        viewModel.RecentProducts = await _context.Products
            .OrderByDescending(p => p.Id)  // Assumes higher Id = newer
            .Take(5)
            .Select(p => new RecentProductViewModel
            {
                Name = p.Name,
                Price = p.Price,
                CategoryName = p.Category.Name,
                DateAdded = DateTime.Now.AddDays(-p.Id) // Fake date for demo
            })
            .ToListAsync();
```

```
        // 4. Top expensive products
        viewModel.TopExpensiveProducts = await _context.Products
            .OrderByDescending(p => p.Price)
            .Take(3)
            .Select(p => new ProductSummaryViewModel
            {
                Id = p.Id,
                Name = p.Name,
                Price = p.Price,
                CategoryName = p.Category.Name
            })
            .ToListAsync();

        // 5. Categories with product counts
        viewModel.CategoriesWithCount = await _context.Categories
            .Select(c => new CategorySummaryViewModel
            {
                Id = c.Id,
                Name = c.Name,
                ProductCount = c.Products.Count
            })
            .ToListAsync();

        return View(viewModel);
    }
}
```

```
@model ProductCatalog.ViewModels.ProductDashboardViewModel

@{
    ViewData["Title"] = "Product Dashboard";
}

<style>
    body {
        background-color: #f8fafc;
        font-family: 'Inter', sans-serif;
    }
    .dashboard-title {
        font-weight: 600;
        color: #334155;
        margin-bottom: 1.5rem;
    }
    .stat-card {
        background: white;
        border: 1px solid #e2e8f0;
        border-radius: 16px;
        padding: 1.2rem;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.02);
        transition: all 0.2s ease;
    }
    .stat-card:hover {
        box-shadow: 0 8px 15px rgba(0,0,0,0.05);
        transform: translateY(-2px);
    }
    .stat-label {
        font-size: 0.85rem;
        text-transform: uppercase;
        letter-spacing: 0.5px;
        color: #94a3b8;
        margin-bottom: 0.3rem;
    }
    .stat-value {
        font-size: 2rem;
        font-weight: 600;
```

No issues found

```
    .stat-label {
        font-size: 0.85rem;
        text-transform: uppercase;
        letter-spacing: 0.5px;
        color: #94a3b8;
        margin-bottom: 0.3rem;
    }
    .stat-value {
        font-size: 2rem;
        font-weight: 600;
        color: #1e293b;
        line-height: 1.2;
    }
    .stat-value small {
        font-size: 1rem;
        color: #64748b;
        font-weight: 400;
    }
    .section-heading {
        font-size: 1.2rem;
        font-weight: 600;
        color: #334155;
        margin: 1.8rem 0 1rem 0;
        padding-bottom: 0.5rem;
        border-bottom: 2px solid #e2e8f0;
    }
    .minimal-table {
        width: 100%;
        border-collapse: collapse;
        background: white;
        border-radius: 12px;
        overflow: hidden;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.02);
        border: 1px solid #e2e8f0;
    }
    .minimal-table th {
        background-color: #f8fafc;
        color: #475569;
```

No issues found



```
.minimal-table th {
    background-color: #f8fafc;
    color: #475569;
    font-weight: 600;
    font-size: 0.8rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    padding: 0.9rem 1rem;
    border-bottom: 1px solid #d1d5db;
}
.minimal-table td {
    padding: 0.8rem 1rem;
    color: #1e293b;
    border-bottom: 1px solid #f1f5f9;
}
.minimal-table tbody tr:last-child td {
    border-bottom: none;
}
.minimal-table tbody tr:hover {
    background-color: #f1f5f9;
}
.badge-soft {
    background-color: #e2e8f0;
    color: #334155;
    padding: 0.25rem 0.7rem;
    border-radius: 20px;
    font-size: 0.8rem;
    font-weight: 500;
}
.category-card {
    background: white;
    border: 1px solid #e2e8f0;
    border-radius: 14px;
    padding: 1.2rem;
    transition: all 0.2s ease;
}
.category-card:hover {
```

No issues found



```
.category-card:hover {
    border-color: #cbd5e1;
    box-shadow: 0 8px 15px rgba(0,0,0,0.03);
}
.category-name {
    font-weight: 600;
    color: #1e293b;
    margin-bottom: 0.4rem;
}
.category-count {
    font-size: 0.9rem;
    color: #64748b;
    display: flex;
    align-items: center;
    gap: 0.4rem;
}
.category-count .count-number {
    background-color: #e2e8f0;
    padding: 0.2rem 0.6rem;
    border-radius: 20px;
    font-weight: 600;
    color: #1e293b;
}
.btn-soft {
    background-color: white;
    border: 1px solid #cbd5e1;
    color: #334155;
    padding: 0.4rem 1rem;
    border-radius: 30px;
    font-size: 0.8rem;
    text-decoration: none;
    transition: all 0.2s;
    display: inline-block;
}
.btn-soft:hover {
    background-color: #f1f5f9;
    border-color: #94a3b8;
```

No issues found

```
    .btn-soft:hover {
        background-color: #f1f5f9;
        border-color: #94a3b8;
        text-decoration: none;
    }
</style>

<h1 class="dashboard-title">Product Dashboard</h1>

<!-- Stat cards -->
<div class="row g-4">
    <div class="col-md-4">
        <div class="stat-card">
            <div class="stat-label">Total Products</div>
            <div class="stat-value">@Model.TotalProducts</div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="stat-card">
            <div class="stat-label">Total Categories</div>
            <div class="stat-value">@Model.TotalCategories</div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="stat-card">
            <div class="stat-label">Average Price</div>
            <div class="stat-value">
                R @Model.AveragePrice.ToString("F2")
            </div>
        </div>
    </div>
</div>

<div class="section-heading">🆕 Recently Added</div>

@if (Model.RecentProducts.Any())
{
```

No issues found

```
@if (Model.RecentProducts.Any())
{
    <table class="minimal-table">
        <thead>
            <tr>
                <th>Product</th>
                <th>Price</th>
                <th>Category</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var product in Model.RecentProducts)
            {
                <tr>
                    <td>@product.Name</td>
                    <td>R @product.Price.ToString("F2")</td>
                    <td>@product.CategoryName</td>
                </tr>
            }
        </tbody>
    </table>
}
else
{
    <p class="text-muted">No recent products.</p>
}

<div class="section-heading">⏱ Most Expensive</div>

@if (Model.TopExpensiveProducts.Any())
{
    <table class="minimal-table">
        <thead>
            <tr>
                <th>Product</th>
                <th>Price</th>
```

No issues found

```
                <th>Price</th>
                <th>Category</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var product in Model.TopExpensiveProducts)
            {
                <tr>
                    <td>@product.Name</td>
                    <td>R @product.Price.ToString("F2")</td>
                    <td>@product.CategoryName</td>
                </tr>
            }
        </tbody>
    </table>
}
else
{
    <p class="text-muted">No products.</p>
}

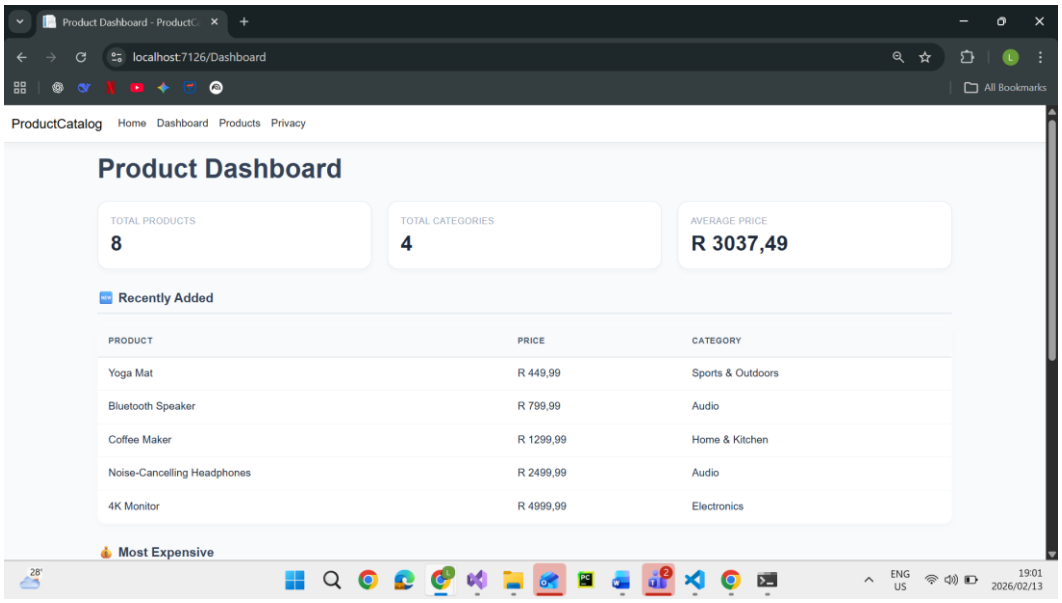
<div class="section-heading">Categories by Product Count</div>

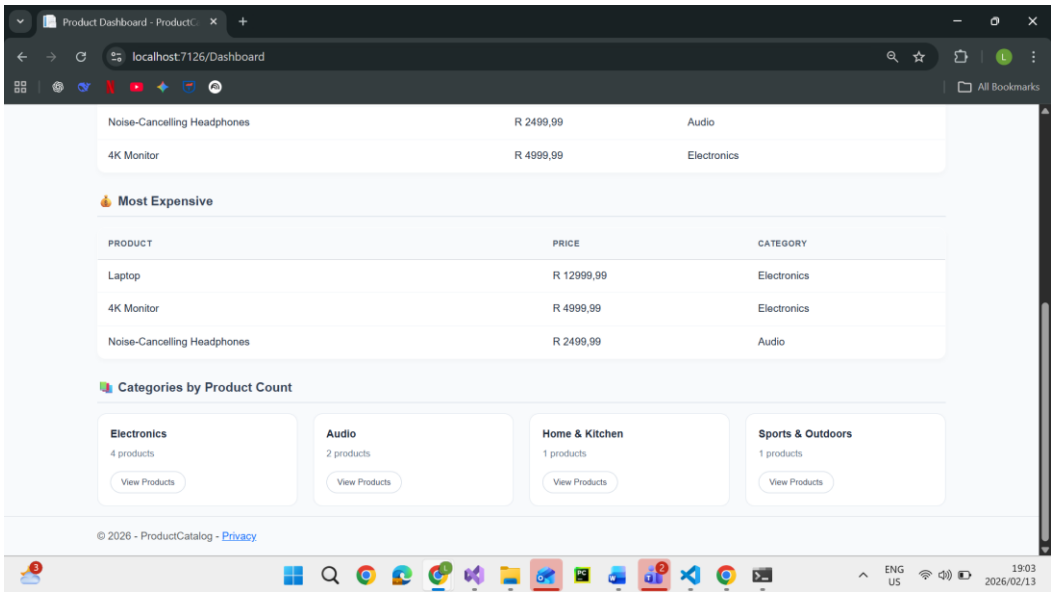
@if (Model.CategoriesWithCount.Any())
{
    <div class="row g-4">
        @foreach (var category in Model.CategoriesWithCount.OrderByDescending(c => c.ProductCount))
        {
            <div class="col-md-3 col-sm-6">
                <div class="category-card">
                    <div class="category-name">@category.Name</div>
                    <div class="category-count">
                        @category.ProductCount products
                    </div>
                    <div class="mt-3">
                        <a class="btn-soft"
                           asp-controller="Product"
```

No issues found

```
                           asp-action="Index"
                           asp-route-categoryId="@category.Id">
                            View Products
                        </a>
                    </div>
                </div>
            </div>
        }
    </div>
}
else
{
    <p class="text-muted">No categories found.</p>
}
```

Output:





💡 ViewModel Best Practices:

Keep ViewModels in a dedicated ViewModels folder.

Name them after the view they serve (e.g., ProductDashboardViewModel).

Use nested ViewModels to keep each piece focused.

Consider AutoMapper to automate mapping between entities and ViewModels.

Never pass domain entities directly to views – always use ViewModels for complex dashboards.

4.9 Chapter Summary

In this chapter, we explored the View layer of ASP.NET Core MVC – how dynamic web pages are built using Razor, how to organise view files, and how to leverage built-in tools to create efficient, maintainable user interfaces. Here's a summary of the concepts we covered:

1. Razor Syntax & Logic

- **The @ Bridge:** Razor allows a seamless mix of C# and HTML, using the @ symbol for inline expressions, code blocks, and control structures like @if and @foreach.
- **Built-in Security:** To protect your application, Razor automatically HTML-encodes output by default to prevent XSS (Cross-Site Scripting) attacks.

2. Data Handling: Models & ViewModels

- **Strongly-Typed Views:** Using the @model directive is the "gold standard," providing IntelliSense and compile-time type checking to reduce runtime errors.
- **ViewModels:** For complex pages, we use ViewModels to combine multiple data sources and shape data specifically for the UI, which helps prevent "over-posting" security risks.
- **Auxiliary Containers:** While ViewBag and ViewData handle simple metadata, TempData is used to persist messages across redirects (essential for the Post-Redirect-Get pattern).

3. Modern UI Tools

- **Tag Helpers:** These modern, HTML-friendly attributes (like <a asp-action="Index">) are the preferred way to generate links and forms, offering better readability than older HTML Helpers.
- **Scaffolding:** We used Visual Studio's code-generation tools to rapidly build functional CRUD (Create, Read, Update, Delete) interfaces for our models.

Having mastered the art of building dynamic interfaces and connecting them to your data logic, you now possess the core tools to create a functional web application. However, a professional application is only complete when it is secure; currently, any user can access, edit, or delete your data. In **Chapter 5: User Authentication and Role-Based Access Control (RBAC)**, we will move beyond public access to implement robust security measures. You will learn how to verify user identities through **Authentication** and how to use **Authorization** to restrict specific actions—like deleting a student or modifying a product—to only those users with the appropriate permissions.

4.10 Review Questions

1. Explain the purpose of the @ symbol in a Razor view. Give three distinct examples of its use (inline expression, code block, and a control structure).
2. Describe the default view discovery path in ASP.NET Core MVC. If a controller named ProductController has an action Details(int id), where will the framework look for the corresponding view? What happens if it isn't found there?
3. What does the @model directive do at the top of a Razor view? List at least three benefits of using strongly-typed views over weakly-typed alternatives like ViewBag.
4. Compare and contrast ViewData, ViewBag, and TempData. Include their lifetimes, syntax, and typical use cases. Which one is most commonly used to pass a SelectList to a dropdown?
5. Define a ViewModel and explain how it differs from a domain model (entity). Give two scenarios where using a ViewModel is preferable to passing domain models directly to a view.
6. Given a collection of Product objects called Model, write a foreach loop that renders an HTML table with columns for product name, price (formatted as Rands), and category name. Include a table header row.
7. You have a controller named AdminController with an action Dashboard. Where should the corresponding view file be placed, and what should it be named? Provide the full relative path.
8. Write a complete form using Tag Helpers that submits to the Create action of the ProductController. Include fields for Name, Price, and a dropdown for CategoryId (assume ViewBag.CategoryList is already populated). Add a submit button and a validation summary.
9. Convert the following Tag Helper form snippet into its HTML Helper equivalent (using Html.BeginForm, Html.LabelFor, Html.TextBoxFor, etc.):

```
<form asp-action="Edit" method="post">
    <label asp-for="Name"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name"></span>
    <button type="submit">Save</button>
</form>
```

10. Design a ViewModel named ProductDashboardViewModel that includes:
    - Total number of products
    - Average price of all products
    - A list of the five most recent products (with name, price, and category name)

- A list of categories with their product counts.
  Write the C# class definition with appropriate nested ViewModel classes.

11. Write an asynchronous controller action that populates the ProductDashboardViewModel you designed above. Use ApplicationDbContext and assume the Product entity has a Category navigation property. Use LINQ projections to fill the lists.

5. User Authentication and Role-Based Access Control (RBAC)

Learning Outcomes:

By the end of this chapter, students should be able to:

- **Evaluate Authentication Types:** Analyze the architectural differences between *Individual Accounts*, *Microsoft Identity Platform*, and *Windows Authentication* to determine the appropriate security model for a project.
- **Implement Role-Based Access Control (RBAC):** Understand the relationship between RoleManager and UserManager in the ASP.NET Core Identity ecosystem.
- **Develop Automated Seeding Logic:** Create a robust RoleSeeder class to programmatically ensure essential roles and administrative accounts exist upon application startup.
- **Configure Dependency Injection Scopes:** Correctly utilize IServiceProvider and CreateScope() within Program.cs to execute asynchronous database seeding during the application bootstrap process.
- **Manage Administrative Defaults:** Implement logic to create a "Super Admin" user with multi-role assignments while maintaining security best practices like email confirmation flags.

When creating a new ASP.NET Core MVC project, the choice you make in the **"Authentication Type"** dialog significantly changes how your application handles user logins and where it stores their data.

Think of it as choosing between building your own vault (**Individual Accounts**), using a global security service (**Microsoft Identity Platform**), or trusting the office keycard system (**Windows**).

5.1 Authentication Types

5.1.1. Individual Accounts (ASP.NET Core Identity)

This is the most common choice for public-facing websites where you want full control over your user list.

- **How it works:** Your application has its own private membership database (managed via Entity Framework). When a user registers, their username and hashed password are stored in **your** SQL Server tables (e.g., AspNetUsers).
- **User Experience:** Users create a local account specifically for your site. You can also "link" social logins like Google or Facebook, but the primary identity still lives in your database.

It is best for independent web apps, startups, or any site where you want to manage users, passwords, and profile data yourself.

5.1.2. Microsoft Identity Platform (Microsoft Entra ID / Azure AD)

This option offloads the entire security responsibility to Microsoft's cloud infrastructure.

- **How it works:** our application does **not** store passwords. Instead, it redirects users to a Microsoft login page (similar to how you sign into Outlook or Teams). Once they log in, Microsoft sends a secure "Token" back to your app.
- **User Experience:** Users log in using their "Work or School" accounts (Office 365) or personal Microsoft accounts.

It is best for enterprise/Internal business applications where employees already have Microsoft accounts, or "SaaS" apps where you want to allow other companies to log in using their own corporate credentials.

5.1.3. Windows Authentication

This is an "invisible" form of authentication designed specifically for private corporate networks (Intranets).

- **How it works:** It relies on the Windows Operating System. When a user opens the site, the browser automatically sends their currently logged-in Windows/Active Directory credentials to the server.
- **User Experience: Zero-touch login.** The user is never prompted for a username or password because they already logged into their computer.

It is best for internal company tools, HR portals, or dashboards that will **only** be accessed by employees on company-managed devices within the same network/domain.

5.2 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security strategy where permissions are assigned to specific **Roles** (like *Admin* or *Student*) rather than individual users. Users are then assigned to one or more of these roles.

The logic is simple:

1. **Define Roles:** "What are the job functions in my app?"
2. **Assign Users:** "What is Lunga's job function?"
3. **Authorize:** "Is Lunga an 'Admin'? If yes, let him delete this record."

This makes managing security much easier as your user base grows. Instead of updating 1,000 individual user permissions, you simply update the "Instructor" role.

5.2.1 ASP.NET Core MVC Identity and RBAC Setup:

To implement this RBAC effectively, we utilize a **Role Seeder class** during the application's startup phase. This approach automates the creation of essential roles and administrative accounts, ensuring that the security infrastructure is consistent and fully functional across all environments – from local development to production. By programmatically "seeding" the database, we eliminate the risk of manual setup errors and guarantee that at least one "Super Admin" exists to manage the system immediately upon deployment, providing a reliable foundation for your RBAC logic.

Step 1: Create the Project with Identity

To use the "Individual Accounts" system, you must select it during project creation so Visual Studio includes the necessary NuGet packages and database contexts.

1. Open Visual Studio and select **Create a new project**.
2. Choose **ASP.NET Core Web App (Model-View-Controller)**.
3. On the **Additional Information** screen:
    - Find the **Authentication Type** dropdown.
    - Select **Individual Accounts**.
4. Click **Create**.

> 💡**What just happened?** Visual Studio automatically added **Microsoft.AspNetCore.Identity.EntityFrameworkCore** and created a **Data** folder containing the ApplicationDbContext.

Step 2: Customize ApplicationUser

Add personal details, plus StudentNumber and StaffNumber to distinguish roles.

- ✓ Open Models/ApplicationUser.cs (create if missing) and extend IdentityUser:

```
26 references
public class ApplicationUser:IdentityUser
{
    5 references
    public string FirstName { get; set; }

    5 references
    public string LastName { get; set; }

    2 references
    public string? StaffNumber { get; set; }

    1 reference
    public string? StudentNumber { get; set; }
}
```

- ✓ Update Data/ApplicationDbContext.cs to use ApplicationUser:

```
12 references
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
```

Step 3: Add Course & Enrollment Models with Data Annotations

Create the domain models and use data annotations for validation. InstructorId is nullable so a course can exist without an instructor initially.

- ✓ Create Models/Course.cs:

```
21 references
public class Course
{
    11 references
    public int Id { get; set; }

    [Required(ErrorMessage = "Title is required")]
    [StringLength(100, MinimumLength = 3)]
    20 references
    public string Title { get; set; }

    [Range(1, 10, ErrorMessage = "Credits must be between 1 and 10")]
    16 references
    public int Credits { get; set; }

    // Nullable foreign key - course can be assigned an instructor later
    14 references
    public string? InstructorId { get; set; }

    // Navigation properties
    10 references
    public ApplicationUser? Instructor { get; set; }
    1 reference
    public ICollection<Enrollment> Enrollments { get; set; } = new List<Enrollment>();
}
```

✓ Create Models/Enrollment.cs:

```
1 reference
public enum Grade { A, B, C, D, F }
19 references
public class Enrollment
{
    11 references
    public int Id { get; set; }

    [Required]
    9 references
    public int CourseId { get; set; }

    [Required]
    10 references
    public string StudentId { get; set; }

    [Display(Name = "Grade")]
    13 references
    public Grade? Grade { get; set; }

    [DataType(DataType.Date)]
    [Display(Name = "Enrollment Date")]
    13 references
    public DateTime EnrollmentDate { get; set; }

    // Navigation properties
    12 references
    public Course Course { get; set; }
    10 references
    public ApplicationUser Student { get; set; }
}
```

Step 4: Update DbContext & Configure Relationships

Add DbSets and configure the relationships, especially handling the nullable Instructor.

- ✓ Modify ApplicationDbContext.cs:

```
12 references
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    13 references
    public DbSet<Course> Courses { get; set; }
    8 references
    public DbSet<Enrollment> Enrollments { get; set; }
    0 references
    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        // Enrollment -> Course (required)
        builder.Entity<Enrollment>()
            .HasOne(e => e.Course)
            .WithMany(c => c.Enrollments)
            .HasForeignKey(e => e.CourseId)
            .OnDelete(DeleteBehavior.Restrict);
```

```
        // Enrollment -> Student (required)
        builder.Entity<Enrollment>()
            .HasOne(e => e.Student)
            .WithMany()
            .HasForeignKey(e => e.StudentId)
            .OnDelete(DeleteBehavior.Restrict);

        // Course -> Instructor (optional, because InstructorId is nullable)
        builder.Entity<Course>()
            .HasOne(c => c.Instructor)
            .WithMany()
            .HasForeignKey(c => c.InstructorId)
            .OnDelete(DeleteBehavior.SetNull); // If instructor is deleted, set InstructorId to null
    }
}
```

Step 5: Scaffold CRUD Controllers & Views

Use Visual Studio's scaffolding to generate fully functional controllers and views for Course and Enrollment.

1. Right-click the **Controllers** folder → **Add** → **New Scaffolded Item**.
2. Choose **MVC Controller with views, using Entity Framework**.
3. For **Model class** select Course, for **Data context class** select ApplicationDbContext.
4. Click **Add**. This generates CoursesController and the corresponding views in Views/Courses/.
5. Repeat for Enrollment.

Note: The generated code includes async actions, validation based on data annotations, and tag helpers. You can now browse to /Courses and /Enrollments to see the CRUD UI.

Customization: You may later modify the EnrollmentsController to restrict students to their own enrollments using the current user's ID.

Improving the Scaffolded Code – Best Practices

The scaffolded code is a great starting point, but you'll often want to tweak it for a better user experience. Here are some common improvements you can apply without making the guide overly long:

- **Display names instead of IDs:** In dropdowns (e.g., selecting a student for an enrollment), use meaningful text like StudentNumber rather than the GUID. Modify the Create and Edit actions in the controller to build the SelectList with the desired display field.
- **Show related data in tables:** For the Courses list, display the instructor's full name instead of the InstructorId. Use the navigation property item.Instructor?.FirstName and item.Instructor?.LastName in the view (with a null check).
- **Cleaner Razor syntax:** Replace raw @item.Property with @Html.DisplayFor(modelItem => item.Property) to respect display templates and formatting. Also, conditionally render Edit/Delete links based on user roles using User.IsInRole().

Viewing the Database in SQL Server Object Explorer

1. In Visual Studio, open **View → SQL Server Object Explorer**
2. In the tree, expand **SQL Server → (localdb)\MSSQLLocalDB** (or the server name from your connection string).
3. Locate your database – by default it is named **SchoolManagementSystemDB** (or whatever you set in the connection string).

4. Expand the database node, then expand **Tables**.
5. To see the data, right-click a table (e.g., dbo.AspNetUsers, dbo.Courses, etc.) and select **View Data**.
6. The table contents will appear in the main window, allowing you to verify that roles, users, and courses have been seeded correctly.

Example improvements in Courses/Index.cshtml:

```
<tbody>
    @foreach (var item in Model) {
        <tr>
            <td>@Html.DisplayFor(modelItem => item.Title)</td>
            <td>@Html.DisplayFor(modelItem => item.Credits)</td>
            <td>@(item.Instructor != null ? $"{item.Instructor.FirstName} {item.Instructor.LastName}" : "Not Assigned")</td>
            <td>
                @if (User.IsInRole("Admin") || User.IsInRole("Instructor"))
                {
                    <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> @:|
                    <a asp-action="Delete" asp-route-id="@item.Id">Delete</a> @:|
                }
                <a asp-action="Details" asp-route-id="@item.Id">Details</a>
            </td>
        </tr>
    }
</tbody>
```

Example Controller change for EnrollmentsController.Create to show student numbers:

```
public class EnrollmentsController : Controller
{
    private readonly ApplicationDbContext _context;
    private readonly UserManager<ApplicationUser> _userManager;

    // Inject UserManager alongside the DbContext
    public EnrollmentsController(ApplicationDbContext context, UserManager<ApplicationUser> userManager)
    {
        _context = context;
        _userManager = userManager;
    }

    [Authorize(Roles = "Admin,Instructor")]
    public async Task<IActionResult> Create()
    {
        ViewData["CourseId"] = new SelectList(_context.Courses, "Id", "Title");

        // Retrieve only users who are assigned the "Student" role
        var students = await _userManager.GetUsersInRoleAsync("Student");

        // Populate the dropdown using the filtered list and display StudentNumber
        ViewData["StudentId"] = new SelectList(students, "Id", "StudentNumber");

        return View();
    }
}
```

ⓘ **Note:** These are just examples – you can apply similar logic to other views and controllers. The goal is to make the UI intuitive and secure. We'll revisit role-based customizations in later steps.

Step 6: Scaffold Identity Views

By default, Identity UI is provided as a library. To customize the pages (Login, Register, etc.), we scaffold them into the project.

- Right-click the project → Add → New Scaffolded Item.
- Select Identity and click Add.
- In the dialog, choose which pages to override (e.g., Account/Register).
- Set Data context class to ApplicationDbContext and User class to ApplicationUser.
- Click Add. Visual Studio creates a Areas/Identity folder with all the selected page views.

ⓘ Why scaffold Identity views? Now you can customize the registration form to include FirstName, LastName, and optionally StudentNumber or StaffNumber based on role selection.

✓ Modify Areas/Identity/Pages/Account/Register.cshtml.cs to add extra fields:

```
1 reference
public class InputModel
{

    [Required]
    [Display(Name = "First Name")]
    3 references
    public string FirstName { get; set; }

    [Required]
    [Display(Name = "Last Name")]
    3 references
    public string LastName { get; set; }

    [Display(Name = "Student Number (if student)")]
    3 references
    public string? StudentNumber { get; set; }

    [Display(Name = "Staff Number (if staff)")]
    3 references
    public string? StaffNumber { get; set; }

    // ... rest of the fields ...
```



Don't forget to update the corresponding Register.cshtml to render the new fields.

```
<form id="registerForm" asp-route-returnUrl="@Model.ReturnUrl" method="post">
    <h2>Create a new account.</h2>
    <hr />
    <div asp-validation-summary="ModelOnly" class="text-danger" role="alert"></div>
    <div class="form-floating mb-3">
        <input asp-for="Input.FirstName" class="form-control" autocomplete="username" aria-required="true" placeholder="Enter First Name" />
        <label asp-for="Input.FirstName">First Name</label>
        <span asp-validation-for="Input.FirstName" class="text-danger"></span>
    </div>
    <div class="form-floating mb-3">
        <input asp-for="Input.LastName" class="form-control" autocomplete="username" aria-required="true" placeholder="Enter Last Name" />
        <label asp-for="Input.LastName">Last Name</label>
        <span asp-validation-for="Input.LastName" class="text-danger"></span>
    </div>
    <div class="form-floating mb-3">
        <input asp-for="Input.StudentNumber" class="form-control" autocomplete="username" aria-required="true" placeholder="Emter Student Number" />
        <label asp-for="Input.StudentNumber">Student Number</label>
        <span asp-validation-for="Input.StudentNumber" class="text-danger"></span>
    </div>
    <div class="form-floating mb-3">
        <input asp-for="Input.StaffNumber" class="form-control" autocomplete="username" aria-required="true" placeholder="Enter Staff Number" />
        <label asp-for="Input.StaffNumber">Staff Number</label>
        <span asp-validation-for="Input.StaffNumber" class="text-danger"></span>
    </div>
```

✓ Update _LoginPartial.cshtml to use ApplicationUser:

Before:

```
@using Microsoft.AspNetCore.Identity
@inject SignInManager<IdentityUser> SignInManager
@inject UserManager<IdentityUser> UserManager

<ul class="navbar-nav">
@if (SignInManager.IsSignedIn(User))
{
    <li class="nav-item">
        <a  class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Manage/Index" title="Manage">Hello @User.Identity?.Name!</a>
    </li>
    <li class="nav-item">
        <form  class="form-inline" asp-area="Identity" asp-page="/Account/Logout" asp-route-returnUrl="@Url.Action("Index", "Home", new { area = "" })">
            <button  type="submit" class="nav-link btn btn-link text-dark">Logout</button>
        </form>
    </li>
}
else
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Register">Register</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Login">Login</a>
    </li>
}
</ul>
```

After:

```
@using Microsoft.AspNetCore.Identity
@inject SignInManager<ApplicationUser> SignInManager
@inject UserManager<ApplicationUser> UserManager

@if (SignInManager.IsSignedIn(User))
{
    var user = await UserManager.GetUserAsync(User);
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-page="/Account/Manage/Index">Hello @user?.FirstName @user?.LastName!</a>
    </li>
    <li class="nav-item">
        <form class="form-inline" asp-area="Identity" asp-page="/Account/Logout" asp-route-returnUrl="@Url.Action("Index", "Home")">
            <button type="submit" class="nav-link btn btn-link">Logout</button>
        </form>
    </li>
}
else
{
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-page="/Account/Register">Register</a>
    </li>
    <li class="nav-item">
        <a class="nav-link" asp-area="Identity" asp-page="/Account/Login">Login</a>
    </li>
}
```

> [i] The **var user = await UserManager.GetUserAsync(User)**; line retrieves the full ApplicationUser object, allowing you to display the user's first and last name instead of just the email/username.

Step 7: Run Migrations

Create and apply the initial migration to build the database schema including our new entities and custom user fields.

✓ Open Package Manager Console (Tools → NuGet Package Manager) and run:

- Add-Migration InitialCreate
- Update-Database

> ⓘ **This creates tables**: AspNetUsers (with our extra columns), AspNetRoles, Courses, Enrollments, and all Identity-related join tables.

Step 8: Create RBAC Role Seeder

Seed the database with roles (Admin, Instructor, Student) and a default Admin user. Optionally seed sample instructors and students.

✓ Create Data/RoleSeeder.cs:

```
1 reference
public class RoleSeeder
{
    1 reference
    public static async Task SeedRolesAndAdminAsync(IServiceProvider serviceProvider)
    {
        using (var scope = serviceProvider.CreateScope())
        {
            var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
            var userManager = scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
            // 1. Create roles
            string[] roleNames = { "Admin", "Instructor", "Student" };
            foreach (var role in roleNames)
            {
                if (!await roleManager.RoleExistsAsync(role))
                    await roleManager.CreateAsync(new IdentityRole(role));
            }
            // 2. Create master Admin
            var adminEmail = "admin@school.com";
            var adminUser = await userManager.FindByEmailAsync(adminEmail);
```

```
            if (adminUser == null)
            {
                adminUser = new ApplicationUser
                {
                    UserName = adminEmail,
                    Email = adminEmail,
                    FirstName = "System",
                    LastName = "Administrator",
                    StaffNumber = "ADM001",  // Staff number for admin
                    EmailConfirmed = true
                };
                var result = await userManager.CreateAsync(adminUser, "Admin@123");
                if (result.Succeeded)
                {
                    await userManager.AddToRoleAsync(adminUser, "Admin");
                    // Admin also has instructor rights
                    await userManager.AddToRoleAsync(adminUser, "Instructor");
                }
            }
        }
    }
}
```

Step 9: Create Sample Data Seeder (Instructors, Students, Courses)

Now we'll create a separate service to seed realistic sample data: instructors, students, and courses. This runs after roles and admin user are seeded, and ensures the database has meaningful content for testing and demonstration.

- ✓ Create a new class Data/SampleDataSeeder.cs with the following code:

```
1 reference
public class SampleDataSeeder
{
    1 reference
    public static async Task SeedAsync(IServiceProvider serviceProvider)
    {
        using (var scope = serviceProvider.CreateScope())
        {
            var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            var userManager = scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
            var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

            // Create Instructors (only if they don't already exist)
            var instructorData = new[]
            {
                new
                {
                    UserName = "LungaAphane@Richfield.ac.za",
                    Email = "LungaAphane@Richfield.ac.za",
                    FirstName = "Lunga",
                    LastName = "Aphane",
                    StaffNumber = "STF-001"
                },
                new
                {
                    UserName = "Akshay@Richfield.ac.za",
                    Email = "Akshay@Richfield.ac.za",
                    FirstName = "Akshay",
                    LastName = "Balalee",
                    StaffNumber = "STF-002"
                }
            };
```

```
foreach (var data in instructorData)
{
    var existingUser = await userManager.FindByEmailAsync(data.Email);
    if (existingUser == null)
    {
        var instructor = new ApplicationUser
        {
            UserName = data.UserName,
            Email = data.Email,
            FirstName = data.FirstName,
            LastName = data.LastName,
            StaffNumber = data.StaffNumber,
            EmailConfirmed = true
        };
        var result = await userManager.CreateAsync(instructor, "Instructor@123");
        if (result.Succeeded)
        {
            await userManager.AddToRoleAsync(instructor, "Instructor");
        }
    }
}
```

```
// Create Students (only if they don't already exist)
var studentData = new[]
{
    new
    {
        UserName = "Benjamin@MyRichfield.ac.za",
        Email = "Benjamin@MyRichfield.ac.za",
        FirstName = "Benjamin",
        LastName = "Montague",
        StudentNumber = "STU-001"
    },
    new
    {
        UserName = "Matilda@MyRichfield.ac.za",
        Email = "Matilda@MyRichfield.ac.za",
        FirstName = "Matilda",
        LastName = "Williams",
        StudentNumber = "STU-002"
    },
    new
    {
        UserName = "Heshlin@MyRichfield.ac.za",
        Email = "Heshlin@MyRichfield.ac.za",
        FirstName = "Heshlin",
        LastName = "Chetty",
        StudentNumber = "STU-003"
    }
};
```

```
foreach (var data in studentData)
{
    var existingUser = await userManager.FindByEmailAsync(data.Email);
    if (existingUser == null)
    {
        var student = new ApplicationUser
        {
            UserName = data.UserName,
            Email = data.Email,
            FirstName = data.FirstName,
            LastName = data.LastName,
            StudentNumber = data.StudentNumber,
            EmailConfirmed = true
        };
        var result = await userManager.CreateAsync(student, "Student@123");
        if (result.Succeeded)
        {
            await userManager.AddToRoleAsync(student, "Student");
        }
    }
}
```

```
        // Create Courses only if none exist (to avoid duplicates)
        if (!await context.Courses.AnyAsync())
        {
            var instructor1 = await userManager.FindByEmailAsync("LungaAphane@Richfield.ac.za");
            var instructor2 = await userManager.FindByEmailAsync("Akshay@Richfield.ac.za");

            var courses = new List<Course>
            {
                new Course { Title = "ASP.NET Core MVC with EF Core", Credits = 5, InstructorId = instructor1?.Id },
                new Course { Title = "Web Technology 511", Credits = 5, InstructorId = instructor1?.Id },
                new Course { Title = "Introduction to Python", Credits = 5, InstructorId = instructor2?.Id },
                new Course { Title = "Machine Learning 600", Credits = 5, InstructorId = instructor2?.Id },
                new Course { Title = "Mathematics 511", Credits = 5, InstructorId = null } // No instructor assigned yet
            };

            await context.Courses.AddRangeAsync(courses);
            await context.SaveChangesAsync();
        }
    }
}
```

ℹ **Note:** This seeder checks for existing users by email before creating them, preventing duplicates on subsequent application starts. It uses the same password for all instructors (Instructor@123) and students (Student@123). You can change these later as needed.

📝 **Important:** The seeder must be called from Program.cs after the role seeder. We'll update Program.cs in the next step to invoke both seeders in the correct order.

Step 10: Configure Program.cs & Call Seeders

Update Identity registration and invoke both RoleSeeder and SampleDataSeeder at application startup.

- ✓ In Program.cs, replace the default Identity registration with:

```
builder.Services.AddDefaultIdentity<ApplicationUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();
```

✓ After var app = builder.Build(); add:

```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    // 1. Seed roles and admin user first
    await RoleSeeder.SeedRolesAndAdminAsync(services);

    // 2. Seed sample instructors, students, and courses
    await SampleDataSeeder.SeedAsync(services);
}
```

📝 **Order matters:** RoleSeeder must run first to create the roles. Then SampleDataSeeder can create users and assign them to those roles.

Step 11: Apply Role-Based Authorization to Controllers

Now that roles and users are seeded, restrict access to controllers and actions using the [Authorize] attribute. This ensures only users with specific roles can perform certain operations.

✓ Add authorization to the CoursesController – only Admins and Instructors can manage courses.

```
// All actions (Index, Details, Create, Edit, Delete) are now restricted to Admin/Instructor.
// If a Student tries to access any of these, they'll be redirected to the Access Denied page.
[Authorize(Roles = "Admin,Instructor")]
1 reference
public class CoursesController ...
```

✓ For EnrollmentsController, create separate actions for different roles:

```
[Authorize]
1 reference
public class EnrollmentsController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public EnrollmentsController(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET: Enrollments
    // Admin and Instructor see all enrollments
    [Authorize(Roles = "Admin,Instructor")]
    3 references
    public async Task<IActionResult> Index()...

    // Students see only their own enrollments
    [Authorize(Roles = "Student")]
    0 references
    public async Task<IActionResult> MyEnrollments()
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var enrollments = await _context.Enrollments
            .Where(e => e.StudentId == userId)
            .Include(e => e.Course)
            .ToListAsync();
        return View(enrollments);
    }
```

```
// GET: Enrollments/Create
// Create and Edit actions are restricted to Admin/Instructor
[Authorize(Roles = "Admin,Instructor")]
0 references
public IActionResult Create()...

// POST: Enrollments/Create
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin,Instructor")]
0 references
public async Task<IActionResult> Create([Bind("Id,CourseId,StudentId,Grade,EnrollmentDate")] Enrollment enrollment)...
```

> ⓘ **Note:** Ensure using Microsoft.AspNetCore.Authorization; and using System.Security.Claims; is added at the top of your controller files.

Step 12: Customize Views Based on User Roles

In your Razor views, you can conditionally render UI elements depending on whether the current user is in a particular role. This improves user experience by showing only relevant options.

✓ In the layout (_Layout.cshtml), show/hide navigation links based on roles:

```
<ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
    </li>
    @if (User.IsInRole("Admin") || User.IsInRole("Instructor"))
    {
        <li class="nav-item">
            <a class="nav-link" asp-controller="Courses" asp-action="Index">Manage Courses</a>
        </li>
        <li class="nav-item">
            <a class="nav-link" asp-controller="Enrollments" asp-action="Index">All Enrollments</a>
        </li>
    }

    @if (User.IsInRole("Student"))
    {
        <li class="nav-item">
            <a class="nav-link" asp-controller="Enrollments" asp-action="MyEnrollments">My Enrollments</a>
        </li>
    }
</ul>
<partial name="_LoginPartial" />
```

✓ In the Courses/Index.cshtml view, show "Edit" and "Delete" buttons only to Admins/Instructors:

```
<table class="table">
    <thead>
        <tr>
            <th>Title</th>
            <th>Credits</th>
            <th>Instructor</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
            <tr>
                <td>@item.Title</td>
                <td>@item.Credits</td>
                <td>@(item.Instructor?.FirstName + item.Instructor?.LastName ?? "Not Assigned")</td>
                <td>
                    @if (User.IsInRole("Admin") || User.IsInRole("Instructor"))
                    {
                        <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> @:|
                        <a asp-action="Delete" asp-route-id="@item.Id">Delete</a> @:|
                    }
                    <a asp-action="Details" asp-route-id="@item.Id">Details</a>
                </td>
            </tr>
        }
    </tbody>
</table>
```

✓ In the Enrollments/MyEnrollments.cshtml view (for students), display only their courses with grades:

```
@model IEnumerable<Enrollment>

<h2>My Enrollments</h2>

<table class="table">
    <thead>
        <tr>
            <th>Course</th>
            <th>Grade</th>
            <th>Enrollment Date</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
            <tr>
                <td>@item.Course.Title</td>
                <td>@(item.Grade?.ToString() ?? "Not graded")</td>
                <td>@item.EnrollmentDate.ToShortDateString()</td>
            </tr>
        }
    </tbody>
</table>
```

📝 **User.IsInRole()** works because we added AddRoles() in Program.cs and seeded roles. The user's roles are automatically loaded when they log in.

5.3 Chapter Summary

This chapter focused on securing ASP.NET Core MVC applications by managing user identities and implementing hierarchical access control. We explored the fundamental concepts of authentication – verifying who a user is – and authorization – determining what they are allowed to do. By the end of this chapter, you should have a solid understanding of how to integrate ASP.NET Core Identity, configure role-based security, and seed initial users and roles for a production-ready application.

1. Authentication Strategies

Choosing the right authentication method is the first critical decision in application security. We examined three common approaches:

- **Individual Accounts** (also known as local authentication) is the standard for web applications that require a private user database stored in SQL Server. The developer manages credentials, password hashing, and user profiles directly using ASP.NET Core

Identity. This approach gives you full control over the user schema and is ideal for custom applications where users register and log in with email and password.

- **Microsoft Identity Platform** (formerly Azure AD) offloads authentication to the cloud. By integrating with Azure Active Directory (now Entra ID), you can allow users to sign in with their existing organizational accounts, support multi-factor authentication, and delegate security maintenance to Microsoft. This is the recommended approach for enterprise applications that need single sign-on (SSO) across multiple services.
- **Windows Authentication** provides seamless SSO for internal corporate networks. Users are automatically authenticated using their Windows domain credentials, making it perfect for intranet applications where all users are part of the same Active Directory domain.

Throughout the chapter, we focused on **Individual Accounts**, as it gives you the most flexibility to customise user data and roles while learning the underlying mechanics of Identity.

2. Role-Based Access Control (RBAC)

RBAC is the industry standard for managing permissions in a scalable way. Instead of assigning rights directly to individual users, we create **roles** (e.g., Admin, Instructor, Student) and then assign users to those roles. Permissions are then granted to roles, not users. This abstraction brings several benefits:

- **Centralised management** – changing a role's permissions affects every user in that role at once.
- **Simplified auditing** – it's easy to see which roles have access to sensitive actions.
- **Clear separation of concerns** – business logic can check roles without knowing the specific users.

In our school management system, we implemented three roles: Admin, Instructor, and Student. This hierarchy allows us to restrict controller actions appropriately – for example, only Admin and Instructor can create or edit courses, while Student can only view their own enrollments.

3. The Role Seeder Class

To make RBAC functional from the very first run of the application, we created a **Role Seeder** – a static utility class that runs at application startup. The seeder performs two essential tasks:

- It checks whether the required roles already exist in the database; if not, it creates them using RoleManager<IdentityRole>.
- It ensures a **master admin account** exists (e.g., admin@school.com) and assigns it the highest privileges (e.g., both Admin and Instructor roles). This guarantees that an

administrator can immediately log in and manage the system without any manual database setup.

The seeder is invoked inside Program.cs after the application is built but before it starts handling requests. Because it runs at startup, the application is "ready-to-use" immediately after deployment – a crucial practice for professional deployments.

4. Implementation Essentials

Several technical details are critical to making RBAC work correctly:

- **Enabling role services in** Program.cs – By default, Identity does not include role support. You must explicitly call .AddRoles<IdentityRole>() when registering Identity services. This adds the necessary stores and managers for roles.
- **Service scoping at startup** – RoleManager and UserManager are scoped services, meaning they cannot be used directly in the static Main method. To seed data, we create a temporary scope using app.Services.CreateScope(), retrieve the required services, and then call our seeder methods.
- **The [Authorize] attribute** – This is the primary tool for enforcing authorization. Placed on a controller or action, it restricts access to authenticated users. When combined with the Roles property (e.g., [Authorize(Roles = "Admin,Instructor")]), it further limits access to users who belong to the specified roles. We demonstrated how to apply this attribute at the controller level (for all actions) and at the action level (for fine-grained control).

5. Putting It All Together

Throughout the chapter, we built a complete school management system that demonstrates:

- Custom ApplicationUser class with additional properties (FirstName, LastName, StudentNumber, StaffNumber).
- Scaffolded CRUD controllers and views for Course and Enrollment.
- A sample data seeder that populates the database with realistic instructors, students, and courses (using our custom email addresses and credentials).
- Customisation of the scaffolded UI to display user-friendly values (e.g., showing instructor names instead of IDs, and using student numbers in dropdowns).
- Role-based customisation of navigation menus and action links in Razor views.

By combining ASP.NET Core Identity, a well-designed role hierarchy, and automated seeding, you can deliver secure, maintainable applications that are ready for real-world use from the moment they are deployed. The patterns established in this chapter – especially the

separation of authentication, role management, and data seeding – form a solid foundation for any enterprise-grade web application

5.4 Review Questions

1. What is the main benefit of RBAC over assigning permissions directly to individual users?
2. In which file do you configure the application to use specific Identity roles, and what method must be added to the Identity service collection?
3. Why should the RoleSeeder check for the existence of a role before attempting to create it?
4. When creating a "Super Admin" via a seeder, why is it common practice to set EmailConfirmed = true?
5. How do you restrict a specific Action method so that only users in the "Instructor" role can access it?
6. What is a "Service Scope" and why is it needed in the Program.cs file for seeding?
7. If a user is assigned to both the "Admin" and "Instructor" roles, and a controller is marked [Authorize(Roles = "Admin")], will they be granted access?
8. What is the difference between applying the [Authorize] attribute at the controller level versus at the individual action method level? Which one takes precedence if both are used?
9. How can you conditionally display UI elements in a Razor view based on the current user's role? Provide a code example showing how to show an "Edit" button only to Admin and Instructor users.
10. Write the complete code snippet from Program.cs that configures Identity to use ApplicationUser, enables roles, and uses ApplicationDbContext as the store.
11. Write the complete CoursesController class header with authorization that restricts all actions to Admin and Instructor roles only. Then show how you would override this for a specific action (e.g., Details) to allow Students to view course details.
12. The default _LoginPartial.cshtml displays "Hello user@email.com". Write the modified code to display "Hello FirstName LastName" instead, assuming you've injected UserManager<ApplicationUser>.

Chapter 6 Extending ASP.NET Core MVC with APIs

Learning Outcomes:

By the end of this chapter, students should be able to:

- Explain what an API is and how it differs from a traditional MVC controller.
- Identify RESTful principles and the appropriate use of HTTP methods (GET, POST, PUT, DELETE) for CRUD operations.
- Create API controllers by inheriting from ControllerBase and applying the [ApiController] attribute.
- Implement full CRUD functionality in an API, returning proper HTTP status codes (200 OK, 201 Created, 404 Not Found, etc.).
- Know about the different ways to test API endpoints using tools like Postman, Swagger, or a browser.
- Consume an API from client-side JavaScript using the Fetch API to build dynamic user interfaces.
- Reuse existing models and database context across both MVC and API controllers to avoid code duplication.
- Understand the basics of API versioning to maintain backward compatibility as your API evolves.

Modern web applications often need to serve data not only to their own views, but also to mobile apps, single-page applications (SPAs), or third-party services. This is where **APIs (Application Programming Interfaces)** come in. In this chapter, you'll learn how to build lightweight, RESTful APIs alongside your MVC application – reusing the same models, business logic, and database context.

6.1 What is an API?

An API is a set of endpoints that accept HTTP requests and return structured data (usually JSON) rather than HTML. Unlike traditional MVC controllers, API controllers:

- Do **not** return views – they return data.
- Use **HTTP status codes** to indicate success or failure (200 OK, 404 Not Found, 400 Bad Request, etc.).

- Rely on **content negotiation** – clients can request JSON or XML (though JSON is the default).

6.2 RESTful Principles

REST (Representational State Transfer) is a popular architectural style for APIs. Key ideas:

- Resources are identified by URLs (e.g., /api/products/5).
- HTTP methods define the operation:
    - GET – retrieve a resource (or list of resources).
    - POST – create a new resource.
    - PUT – update an existing resource (full update).
    - PATCH – partially update a resource.
    - DELETE – remove a resource.
- Stateless – each request contains all information needed; the server does not store client state between requests.

6.3 Creating an API Controller

In ASP.NET Core, you can create a dedicated API controller by inheriting from ControllerBase (instead of Controller). ControllerBase provides helper methods like Ok(), NotFound(), BadRequest(), CreatedAtAction(), etc., but **not** view-related methods like View().

Example: A simple ProductsController

```
[Route("api/[controller]")]
[ApiController]
public class ProductsController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public ProductsController(ApplicationDbContext context)
    {
        _context = context;
    }

    // GET: api/products
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> GetProducts()
    {
        return await _context.Products.ToListAsync();
    }
```

```
    // GET: api/products/5
    [HttpGet("{id}")]
    public async Task<ActionResult<Product>> GetProduct(int id)
    {
        var product = await _context.Products.FindAsync(id);

        if (product == null)
            return NotFound();

        return product;
    }

    // POST: api/products
    [HttpPost]
    public async Task<ActionResult<Product>> PostProduct(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();

        // Return a 201 Created response with a Location header pointing to the new resource
        return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
    }
```

```
    // PUT: api/products/5
    [HttpPut("{id}")]
    public async Task<IActionResult> PutProduct(int id, Product product)
    {
        if (id != product.Id)
            return BadRequest();

        _context.Entry(product).State = EntityState.Modified;
        await _context.SaveChangesAsync();

        return NoContent(); // 204 No Content is standard for a successful PUT
    }

    // DELETE: api/products/5
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteProduct(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null)
            return NotFound();

        _context.Products.Remove(product);
        await _context.SaveChangesAsync();

        return NoContent();
    }
}
```

Key points:

- [ApiController] attribute enables automatic model validation, binding source inference, and other API-specific behaviours.
- [Route("api/[controller]")] sets the base route (e.g., /api/products).
- Return types: ActionResult<T> allows you to return either the object or a status result (like NotFound).

6.3.1 Testing Your API

Once you've built your API endpoints, you need to verify they work correctly. ASP.NET Core makes it easy to test APIs using a variety of tools, from browser-based testing to dedicated API clients.

Testing with a Browser (GET requests only)

For simple GET requests, you can use any web browser. Simply navigate to the API endpoint URL, and the browser will display the JSON response.

Example: If your application is running locally on port 5000, visit:

https://localhost:5001/api/products

The browser will show the JSON array of products. However, browsers can only test GET requests – you cannot send POST, PUT, or DELETE requests this way.

> ⓘ **Note:** Some browsers format JSON nicely by default (Firefox), while others may prompt you to download the file (Chrome). You can install JSON formatter extensions for better readability.

Testing with Postman

Postman is a popular desktop application for testing APIs. It allows you to send any HTTP method, set headers, attach bodies, and inspect responses in a clean interface.

Basic steps to test with Postman:

1. Launch Postman and click **New** → **HTTP Request**.
2. Set the HTTP method (GET, POST, PUT, DELETE) from the dropdown.
3. Enter the full URL (e.g., https://localhost:5001/api/products).
4. For POST and PUT requests:

- Go to the **Body** tab.
- Select **raw** and choose **JSON** from the dropdown.
- Enter your JSON data, e.g.:

```
{
  "title": "New Product",
  "price": 29.99,
  "categoryId": 1
}
```

5. Click **Send**.
6. Inspect the response status, body, and headers at the bottom.

Testing different scenarios:

- **GET** /api/products – should return a 200 OK with a list of products.
- **GET** /api/products/5 – should return a single product or 404 if not found.
- **POST** /api/products – should return 201 Created with a location header.
- **PUT** /api/products/5 – should return 204 No Content on success.
- **DELETE** /api/products/5 – should return 204 No Content on success.

Testing with Swagger (OpenAPI)

Swagger (now part of the OpenAPI initiative) automatically generates interactive documentation for your API. It provides a web-based UI where you can both view endpoint details and execute test requests.

To add Swagger to your project:

1. Install the NuGet package: Swashbuckle.AspNetCore
2. In Program.cs, add Swagger services **before** builder.Build():

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

3. In the app configuration section (after app = builder.Build()), add:

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

4. Run your application and navigate to: https://localhost:5001/swagger
   You'll see a beautiful, interactive documentation page listing all your API endpoints. Each endpoint shows:

   - The HTTP method and route.
   - Expected parameters and request body format.
   - Possible response codes and schemas.

To test an endpoint directly from Swagger:

- Click on an endpoint to expand it.
- Click **Try it out**.
- Fill in any required parameters.
- Click **Execute**.
- View the full response, including status code, headers, and body.

> 💡**Pro Tip:** Swagger is invaluable during development and for sharing API documentation with front-end developers or external partners. It ensures your API is self-documenting and always up-to-date.

6.3.2 Consuming the API from Client-Side JavaScript

One common reason to build an API is to feed data to a front-end framework like React, Vue, or plain JavaScript. Here’s a simple example using the **Fetch API** to retrieve products and display them:

```
<script>
    async function loadProducts() {
        const response = await fetch('/api/products');
        const products = await response.json();

        const list = document.getElementById('product-list');
        products.forEach(p => {
            const item = document.createElement('li');
            item.textContent = `${p.name} - R${p.price}`;
            list.appendChild(item);
        });
    }

    window.addEventListener('load', loadProducts);
</script>

<ul id="product-list"></ul>
```

6.3.3 Sharing Models and Logic

One of the most powerful advantages of building an API inside your existing MVC project is the ability to **reuse your domain models, database context, and business logic**. This approach eliminates duplication, reduces maintenance overhead, and ensures consistency between your web UI and your API.

The Same Model, Two Purposes

Consider the Product entity we defined earlier in the MVC application:

```
public class Product
{
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    public string Title { get; set; }

    [Column(TypeName = "decimal(18,2)")]
    public decimal Price { get; set; }

    public int CategoryId { get; set; }
    public Category Category { get; set; }
}
```

This same model can be used in two completely different ways:

In MVC Controllers	In API Controllers
✓ Passed to views using return View(product)	✓ Returned as JSON using return Ok(product)

In MVC Controllers	In API Controllers
✓ Used in forms with Tag Helpers (asp-for)	✓ Serialized automatically by the framework
✓ Validated with data annotations	✓ Same validation rules apply automatically
✓ Displayed in tables with DisplayNameFor	✓ Used as the schema for Swagger documentation

Shared DbContext and Services

Your ApplicationDbContext is registered once in Program.cs and can be injected into **both** MVC and API controllers:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
```

Then, any controller – whether it inherits from Controller (MVC) or ControllerBase (API) – can request the same context:

```
// MVC Controller (returns Views)
public class ProductsMvcController : Controller
{
    private readonly ApplicationDbContext _context;
    public ProductsMvcController(ApplicationDbContext context) => _context = context;
}

// API Controller (returns JSON)
[ApiController]
[Route("api/[controller]")]
public class ProductsApiController : ControllerBase
{
    private readonly ApplicationDbContext _context;
    public ProductsApiController(ApplicationDbContext context) => _context = context;
}
```

Both controllers now share the exact same data access logic, connection pooling, and transaction behavior.

6.3.4 API Versioning

As your API evolves, you may need to introduce breaking changes. **API versioning** allows you to maintain multiple versions simultaneously. Add the Microsoft.AspNetCore.Mvc.Versioning package and configure it in Program.cs. Then you can specify versions via URL, query string, or header.

Example with URL path:

```
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/[controller]")]
```

6.4 Chapter Summary

In this chapter, you learned how to extend your ASP.NET Core MVC application with a lightweight API. You discovered that API controllers return data—typically JSON—rather than HTML views, and they follow RESTful conventions. By creating a ProductsController that inherits from ControllerBase and using the [ApiController] attribute, you implemented a full set of CRUD endpoints with proper HTTP status codes. You also explored how to test these endpoints using tools like Postman and Swagger, and how to consume them from client-side JavaScript with the Fetch API.

One of the key benefits of building the API inside your MVC project is the ability to reuse your existing models, DbContext, and business logic – keeping your codebase efficient and maintainable. You were also introduced to API versioning, a technique that allows you to introduce changes to your API without breaking existing clients.

With these skills, you can now build applications that serve both interactive web pages and data to mobile apps, single-page applications, or external partners. The API knowledge you've gained here forms a foundation for more advanced topics like authentication for APIs, rate limiting, and integration with cloud services.

6.5 Review Questions

1. **API Definition** – What is an API, and how does it differ from a traditional MVC controller in terms of the type of response it returns?
2. **MVC vs. API Controllers** – List three key differences between an MVC controller (inheriting from Controller) and an API controller (inheriting from ControllerBase).
3. **HTTP Methods** – For each CRUD operation (Create, Read, Update, Delete), identify the corresponding HTTP method (GET, POST, PUT, DELETE) that should be used in a RESTful API.
4. **HTTP Status Codes** – What HTTP status code would you return in the following scenarios:
    - Successfully retrieving a resource?
    - Creating a new resource?
    - A resource not being found?
    - A client sending invalid data?
5. **The [ApiController] Attribute** – What three benefits does the [ApiController] attribute provide when applied to an API controller?
6. **RESTful Principles** – Explain what it means for an API to be "stateless." Why is statelessness important in RESTful design?
7. **ActionResult<T>** – Why does an API action often return ActionResult<T> instead of just T? What flexibility does this provide?
8. **Testing APIs** – Name two tools you can use to test API endpoints without building a client interface. Briefly describe how each works.
9. **Client-Side Consumption** – Write a short JavaScript function using the Fetch API that retrieves a list of products from /api/products and logs them to the console.
10. **API Versioning** – Why might you need to version an API? Give an example scenario where versioning would be necessary to avoid breaking existing clients.